UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática

TFG del Grado en Ingeniería Informática

título del TFG
Documentación Técnica

Presentado por nombre alumno
en Universidad de Burgos — 14 de mayo de 2026
Tutor: nombre tutor

Índice general

Índice general **i**

Índice de figuras **iii**

Índice de tablas **iv**

Apéndice A Plan de Proyecto Software **1**
A.1. Introducción 1
A.2. Planificación temporal 2
A.3. Estudio de viabilidad 8

Apéndice B Especificación de Requisitos **11**
B.1. Introducción 11
B.2. Objetivos generales 11
B.3. Catálogo de requisitos 12
B.4. Especificación de requisitos 16

Apéndice C Especificación de diseño **31**
C.1. Introducción 31
C.2. Diseño de datos 31
C.3. Diseño arquitectónico 34
C.4. Diseño procedimental 38
C.5. Resumen del stack tecnológico 43

Apéndice D Documentación técnica de programación **45**
D.1. Introducción 45
D.2. Estructura de directorios 45

D.3. Manual del programador 47
D.4. Compilación, instalación y ejecución del proyecto 50
D.5. Pruebas del sistema 53

Apéndice E Documentación de usuario 57
E.1. Introducción 57
E.2. Requisitos de usuarios 57
E.3. Instalación 59
E.4. Manual del usuario 61

Apéndice F Anexo de sostenibilización curricular 69
F.1. Introducción 69

Bibliografía 75

Índice de figuras

B.1. Diagrama de casos de uso del paciente. 20
B.2. Diagrama de casos de uso del administrador. 25

C.1. Relaciones lógicas entre entidades del sistema 34
C.2. Diagrama arquitectónico de módulos y dependencias 35
C.3. Arquitectura de despliegue del prototipo 37
C.4. Diagrama de secuencia - Triaje básico de síntomas 39
C.5. Diagrama de secuencia - Añadir síntoma desde administración . 40
C.6. Diagrama de secuencia - Regeneración de un modelo 41
C.7. Diagrama de secuencia - Acceso al panel de administración . . . 42

E.1. Pantalla principal del paciente 62
E.2. Ampliación de síntomas tras una predicción de baja confianza . 64
E.3. Selección manual de especialidad tras agotar las rondas 65
E.4. Configuración del umbral de confianza y de las rondas de ampliación 66
E.5. Revisión y reasignación manual de casos de baja confianza . . . 67

Índice de tablas

A.1. Resumen de Sprints del proyecto 2
A.2. Coste de personal 8
A.3. Coste de recursos hardware (amortizado a 4 meses) 9
A.4. Coste total del proyecto 9
A.5. Licencias de las tecnologías utilizadas 10

B.1. A01 - Paciente. 17
B.2. A02 - Administrador. 18
B.3. A03 - Motor de clasificación. 19
B.4. A04 - Almacenamiento local. 20
B.5. CU-01 Introducir síntomas. 21
B.6. CU-02 Obtener especialidad recomendada. 22
B.7. CU-03 Resolver caso de baja confianza. 23
B.8. CU-04 Reportar error sobre predicción aceptada. 24
B.9. CU-05 Acceder a la pestaña de administración. 25
B.10. CU-06 Configurar el flujo de triaje. 26
B.11. CU-07 Añadir síntomas y especialidad. 27
B.12. CU-08 Revisar y resolver casos del log. 28
B.13. CU-09 Regenerar modelos. 29

C.1. Ficheros y carpetas con datos persistentes 33
C.2. Resumen del stack tecnológico del prototipo 43

Apéndice A

Plan de Proyecto Software

A.1. Introducción

La gestión de este proyecto software ha seguido una metodología ágil, organizando el trabajo en *Sprints* que permiten un desarrollo incremental y adaptativo. Dado que el proyecto se desarrolla de forma individual como Trabajo de Fin de Grado, la metodología adoptada es una variante simplificada de *Scrum*: se mantienen los Sprints como unidad de planificación y entrega, pero se prescinde de roles adicionales y de ceremonias formales como *daily standups*.

El seguimiento de tareas se ha realizado mediante un tablero Kanban gestionado a través de la plataforma *Zube.io* [9], integrada con el repositorio de GitHub. Las tarjetas se organizan en los estados propios de Zube.io: *Inbox* (entrada por defecto), *Backlog*, *Ready*, *In Progress*, *In Review* y *Done*, permitiendo un control visual del avance de cada Sprint. No se han utilizado Epics; en su lugar, los Sprints agrupan directamente las tareas relacionadas con cada componente del sistema.

El proyecto se articula en torno a cuatro bloques técnicos de modelado —*Term Frequency – Inverse Document Frequency* (TF-IDF) como línea base, TF-IDF con reducción *Singular Value Decomposition* (SVD), un clasificador *Support Vector Machine* (SVM) y una Red Neuronal Artificial (RNA) de tipo *Multi-layer Perceptron* (MLP)— a los que se añaden las fases de preparación de datos, desarrollo del frontal de usuario, panel de administración y documentación. La memoria y los anexos se han elaborado de forma paralela al desarrollo.

Sprint	Inicio	Fin
Introducción y Objetivos	19/02/2026	03/03/2026
Manejo y Edición con TeXTStudio	19/02/2026	03/03/2026
Búsqueda y conversión de DataSheets con datos	26/02/2026	23/04/2026
Creación de Script TF-IDF	01/04/2026	23/04/2026
Creación de Front End	21/04/2026	23/04/2026
Creación de Script TFIDF-SVD	22/04/2026	06/05/2026
Creación de Script SVM	26/04/2026	06/05/2026
Teoría del TFG — Memoria	03/03/2026	06/06/2026
Trabajos sobre el Front END	22/04/2026	01/06/2026
Adaptaciones de Código y Nuevas Funcionalidades	05/05/2026	01/06/2026
Anexos del TFG	06/05/2026	01/06/2026

Tabla A.1: Resumen de Sprints del proyecto

A.2. Planificación temporal

Visión general de los Sprints

A continuación se recoge la relación completa de Sprints realizados, con sus fechas de inicio y fin:

Detalle de los Sprints

Sprint 1 — Introducción y Objetivos

Periodo: 19 de febrero de 2026 – 3 de marzo de 2026.

Sprint inaugural dedicado a establecer la base documental del proyecto. Se definió la estructura de la memoria en LaTeX y se redactó el capítulo de introducción —incluyendo el análisis de la crisis de accesibilidad sanitaria, el estado del arte y la justificación tecnológica— así como la formulación de los objetivos generales, funcionales y no funcionales. El cierre se produjo tres días después de la fecha planificada, motivado por la curva de aprendizaje inicial con el entorno de documentación.

Sprint 2 — Manejo y Edición con TeXTStudio

Periodo: 19 de febrero de 2026 – 3 de marzo de 2026.

Sprint paralelo al primero, dedicado a la adquisición de competencia en TeXTStudio como herramienta principal de documentación técnica. Se trabajó en el manejo de abreviaturas y glosarios, la gestión de referencias bibliográficas con BibTEX, la inserción de figuras y tablas, y la compilación del documento completo. El cierre se adelantó dos días respecto a la fecha planificada.

Sprint 3 — Búsqueda y Conversión de DataSheets con Datos

Periodo: 26 de febrero de 2026 – 23 de abril de 2026.

Sprint dedicado a la localización, evaluación y transformación del conjunto de datos clínicos, en línea con el objetivo funcional de **preparación de los datos**. Los datos de partida consisten en un CSV con tres columnas: descripción de síntomas, enfermedad asociada y especialidad médica. Las tareas realizadas fueron:

- *Búsqueda de Datasheets:* exploración y evaluación de fuentes de datos clínicos públicas con licencia adecuada para el entrenamiento del clasificador.
- *Formato y Traducción:* normalización del formato de los datos, traducción y homogeneización de etiquetas de categorías médicas.
- *Aclaración de fuentes:* documentación de las fuentes seleccionadas, criterios de inclusión/exclusión y características del conjunto de datos definitivo.

Este sprint supuso la mayor desviación temporal del proyecto, cerrándose con casi siete semanas de retraso sobre lo planificado, debido a la escasez de *datasets* clínicos en español con las características requeridas.

Sprint 4 — Creación de Script TF-IDF

Periodo: 1 de abril de 2026 – 23 de abril de 2026.

Sprint centrado en el desarrollo del primer clasificador, que actúa como línea base dentro del objetivo funcional de **clasificación con cuatro modelos**. Se utilizó scikit-learn [5] como librería principal. Las tareas realizadas fueron:

- *Preparación del Entorno:* configuración del entorno de desarrollo Python con las dependencias necesarias (scikit-learn, pandas, numpy).

- *Pruebas en Google Colab:* validación inicial del *pipeline* de Procesamiento de Lenguaje Natural (PLN) y del vectorizador TF-IDF en un entorno de experimentación rápida.
- *Programación:* implementación completa del *pipeline* de preprocesado (normalización, eliminación de *stopwords*, tokenización y generación de bigramas y trigramas) y del clasificador TF-IDF base.
- *A Python y Streamlit tf_idf:* integración del script de clasificación TF-IDF con el entorno de ejecución local y primeras pruebas con Streamlit [8].
- *Separar el módulo que genera el modelo:* refactorización del código para desacoplar la generación y persistencia del modelo de la lógica de predicción, en línea con el objetivo no funcional de **modularidad**.

El sprint se cerró un día antes de lo planificado.

Sprint 5 — Creación de Front End

Periodo: 21 de abril de 2026 – 23 de abril de 2026.

Sprint dedicado al desarrollo de la interfaz de usuario con Streamlit [8], en línea con el objetivo funcional de **predicción a partir de texto libre**:

- *Creación de app.py:* desarrollo del frontal que permite al paciente introducir síntomas por escrito y recibir la especialidad médica predicha junto con el porcentaje de confianza. Se incorporó también la lógica de umbral: cuando la confianza queda por debajo del valor configurado, el sistema solicita al paciente que amplíe la descripción o elige entre las alternativas más probables.

El sprint se cerró con cinco días de antelación respecto a la fecha planificada.

Sprint 6 — Creación de Script TFIDF-SVD

Periodo: 22 de abril de 2026 – 6 de mayo de 2026.

Sprint dedicado al segundo clasificador del objetivo funcional de **clasificación con cuatro modelos**: TF-IDF combinado con reducción de dimensionalidad mediante TruncatedSVD, implementado con scikit-learn [5]. Las tareas realizadas fueron:

- *Separar generar Módulo:* desacoplamiento del módulo de generación del modelo SVD para garantizar la persistencia de transformaciones y la consistencia entre entrenamiento y predicción.

- *Implementación en el app.py:* integración del modelo TF-IDF+SVD en el frontal Streamlit, permitiendo seleccionar el modelo de clasificación activo.

- *Creación del Script:* implementación de TruncatedSVD sobre la matriz TF-IDF, selección del número de componentes mediante validación cruzada y evaluación del rendimiento (F1 macro/micro, tiempo de entrenamiento y consumo de memoria) frente al modelo base.

El sprint se cerró con una semana de retraso sobre lo planificado, debido al tiempo requerido para el análisis sistemático de la varianza explicada en función del número de componentes.

Sprint 7 — Creación de Script SVM

Periodo: 26 de abril de 2026 – 6 de mayo de 2026.

Sprint dedicado al tercer clasificador del objetivo funcional de **clasificación con cuatro modelos**: un SVM lineal entrenado con scikit-learn [5]. Las tareas realizadas fueron:

- *Creación:* implementación del clasificador SVM sobre las representaciones TF-IDF y TF-IDF+SVD, con búsqueda de hiperparámetros (`C`, `class_weight`) mediante `GridSearchCV` y validación cruzada estratificada optimizando macro-F1.

- *Implementación en el Front End:* integración del modelo SVM en el frontal Streamlit como opción de clasificación seleccionable.

- *Separar Generar Modelo:* refactorización para desacoplar la generación y persistencia del modelo SVM del módulo de predicción.

El sprint se cerró cuatro días antes de la fecha planificada.

Sprint 8 — Teoría del TFG — Memoria

Periodo: 3 de marzo de 2026 – 6 de junio de 2026.

Sprint de larga duración, planificado en paralelo con el desarrollo, que cubre la redacción del cuerpo teórico de la memoria. Las tareas contempladas son:

- *Búsqueda de Teoría en la red:* recopilación y análisis de fuentes bibliográficas sobre PLN, clasificación de texto clínico, TF-IDF, SVD, SVM y MLP.
- *Conceptos Teóricos:* redacción del capítulo de fundamentos teóricos de la memoria.
- *Añadir Referencias Teoría:* incorporación y revisión de las referencias bibliográficas al documento.
- *Convertir a LaTeX:* conversión y maquetación del contenido teórico al formato LaTeX de la plantilla del TFG.
- *Técnicas y Herramientas:* redacción del capítulo de técnicas y herramientas utilizadas.
- *Aspectos relevantes del desarrollo del proyecto:* redacción del capítulo de aspectos relevantes del desarrollo.
- *Trabajos Relacionados:* análisis y redacción del capítulo de trabajos relacionados y estado del arte.
- *Conclusiones y Líneas de trabajo Futuras:* redacción del capítulo de conclusiones y trabajo futuro.

Sprint 9 — Trabajos sobre el Front END

Periodo: 22 de abril de 2026 – 1 de junio de 2026.

Sprint que extiende el frontal Streamlit [8] con la integración progresiva de los modelos y la mejora de la experiencia de usuario. Las tareas contempladas son:

- *Inclusión TFIDF:* integración del modelo TF-IDF en la interfaz como opción de clasificación seleccionable.
- *Inclusión SVD:* integración del modelo TF-IDF+SVD en la interfaz.
- *Inclusión SVM en Front End:* integración del clasificador SVM en la interfaz.

- *Añadir Síntomas - Especialidad en Front End:* mejora de la interfaz para mostrar la especialidad médica asociada a cada predicción, incluyendo el indicador de confianza y la lógica de segunda vuelta cuando la predicción queda por debajo del umbral configurado.

Sprint 10 — Adaptaciones de Código y Nuevas Funcionalidades

Periodo: 5 de mayo de 2026 – 1 de junio de 2026.

Sprint que incorpora el cuarto clasificador y el panel de administración, completando así los objetivos funcionales de **clasificación con cuatro modelos** y **panel de administración**. Las tareas contempladas son:

- *Creación del Módulo Red Neuronal:* implementación del cuarto clasificador, una RNA de tipo MLP entrenada con *early stopping* y regularización para evitar sobreajuste. Este modelo completa el conjunto de cuatro enfoques comparados bajo el mismo protocolo de evaluación.
- *Creación Módulo Administrador:* desarrollo del panel de administración con tres herramientas: añadir nuevos pares síntoma-especialidad a un CSV ampliado, revisar y etiquetar el log de consultas no clasificadas con confianza suficiente, y disparar el reentrenamiento de los cuatro modelos con los datos actualizados.
- *Securizar el Módulo Administrador:* implementación de mecanismos de autenticación y control de acceso al panel de administración para restringir su uso al perfil autorizado.

Sprint 11 — Anexos del TFG

Periodo: 6 de mayo de 2026 – 1 de junio de 2026.

Sprint dedicado íntegramente a la redacción de los anexos de la memoria. Las tareas contempladas son:

- *Plan de Proyecto:* redacción del presente apéndice A.
- *Especificación de Requisitos:* redacción del apéndice B con el catálogo de requisitos funcionales y no funcionales.
- *Diseño:* redacción del apéndice C con la especificación de diseño arquitectónico y de datos.

- *Manual del Programador:* redacción del apéndice D con la documentación técnica de programación.
- *Manual del Usuario:* redacción del apéndice E con la guía de instalación y uso de la aplicación.
- *Anexo de sostenibilización curricular:* redacción del apéndice F con la reflexión sobre sostenibilidad.

A.3. Estudio de viabilidad

Viabilidad económica

El proyecto se ha desarrollado íntegramente sobre tecnologías de software libre y código abierto, por lo que el coste de licencias es nulo. A continuación se simula el coste real que supondría el desarrollo en un contexto empresarial.

Costes de personal

Se simula la contratación de un ingeniero informático junior durante los aproximadamente 4 meses de desarrollo efectivo (febrero–mayo de 2026). El salario de referencia se ha obtenido del XVIII Convenio Colectivo Estatal de Empresas de Consultoría y Tecnologías de la Información [3], y los tipos de cotización corresponden a los publicados por la Seguridad Social [6] y la Agencia Tributaria [1] para el ejercicio 2026.

Concepto	**Importe**
Salario mensual bruto	1.800,00 €
Retención IRPF (13,55 %)	243,90 €
Cotización SS empresa (36,25 %)	652,50 €
Coste mensual para la empresa	**2.452,50 €**
Total 4 meses	**9.810,00 €**

Tabla A.2: Coste de personal

Costes de recursos hardware

Se considera únicamente el equipo de trabajo del desarrollador (trabajo desde domicilio), calculando el coste amortizado para los 4 meses de duración del proyecto.

Concepto	Precio sin IVA	Coef.	Coste 4 meses
Ordenador portátil	750,00 €	26 %	65,00 €
Monitor adicional	160,00 €	30 %	16,00 €
Teclado y ratón	30,00 €	30 %	3,00 €
Total	**940,00 €**	—	**84,00 €**

Tabla A.3: Coste de recursos hardware (amortizado a 4 meses)

Coste total del proyecto

Concepto	Coste
Coste de personal (4 meses)	9.810,00 €
Coste de hardware (amortizado)	84,00 €
Herramientas software	0,00 €
TOTAL	**9.894,00 €**

Tabla A.4: Coste total del proyecto

Beneficios y modelo de explotación

Aunque el prototipo se desarrolla como TFG sin ánimo de lucro inmediato, se identifican dos vías de explotación potencial en un contexto empresarial:

- **Servicio SaaS para centros de salud:** la aplicación se ofrecería como servicio alojado con tarifas por volumen de consultas clasificadas (p. ej., cuota mensual por centro).
- **Solución standalone con soporte:** distribución del sistema empaquetado (p. ej., contenedor Docker) con diferentes niveles de soporte técnico y actualización de modelos con los datos propios de cada centro.

Viabilidad legal

El proyecto se apoya exclusivamente en librerías de código abierto con licencias compatibles entre sí y con el uso académico y comercial. Además, al ejecutarse en local sin enviar datos a servicios externos, el sistema es

compatible por diseño con los principios de minimización y soberanía del dato del **Reglamento General de Protección de Datos (RGPD)**.

Tecnología	**Licencia**
Python 3.11	PSF License (permisiva, uso comercial permitido)
scikit-learn, numpy, pandas [5]	BSD 3-Clause (permisiva, uso comercial permitido)
Streamlit [8]	Apache License 2.0 (permisiva, uso comercial permitido)
LaTeX / TeXTStudio	LPPL / GPL (libre para uso académico y personal)
Zube.io [9]	Propietaria, plan gratuito para proyectos individuales
Git	GPL v2 (libre)

Tabla A.5: Licencias de las tecnologías utilizadas

La licencia más restrictiva del conjunto es la GPL, presente solo en herramientas de desarrollo que no forman parte del artefacto distribuible. El código fuente de la aplicación puede distribuirse bajo licencia **MIT**, al no incorporar componentes GPL en el producto final.

En cuanto a los datos, el conjunto utilizado para el entrenamiento procede de fuentes públicas con licencias que permiten su uso en investigación y proyectos académicos. Para un despliegue en producción con datos reales de pacientes, sería necesario cumplir adicionalmente con la **Ley Orgánica de Protección de Datos (LOPD-GDD)**, garantizando el consentimiento informado y la anonimización de los registros clínicos.

Apéndice B

Especificación de Requisitos

B.1. Introducción

Este anexo recoge la especificación de requisitos del prototipo standalone desarrollado para clasificar síntomas en lenguaje natural y asignar la especialidad médica correspondiente. La aplicación se ejecuta en local, sin dependencias de servicios externos, y aborda el problema del triaje y la asignación de citas planteado en la introducción de la memoria.

El documento sirve como referencia funcional durante el análisis y el diseño, y también como base para comprobar el cumplimiento de los requisitos en las fases de validación y pruebas.

La aplicación tiene dos pestañas. La del **paciente** permite escribir los síntomas en texto libre y devuelve la especialidad predicha junto con un indicador de confianza. La del **administrador**, protegida por contraseña, sirve para gestionar el catálogo de síntomas, ajustar la configuración del triaje, revisar los casos no resueltos y reentrenar los cuatro modelos: TF-IDF, TF-IDF con reducción mediante SVD, SVM y una RNA de tipo MLP.

B.2. Objetivos generales

Los objetivos generales del proyecto se recogen en el capítulo 2 de la memoria y son los que han guiado los requisitos de este anexo:

- **Clasificación automática de síntomas en lenguaje natural:** a partir de la descripción libre del paciente, devolver la especialidad médica más adecuada y un indicador de confianza.

- **Flujo de PLN de principio a fin:** cubrir en un único ejecutable la validación de datos, el preprocesado y la predicción.
- **Comparativa de cuatro modelos:** entrenar y evaluar TF-IDF como línea base, TF-IDF con reducción mediante SVD, SVM y un Perceptrón Multicapa, midiendo el acierto y también el coste en tiempo y memoria.
- **Estrategia multimodelo en producción:** ejecutar los cuatro modelos en paralelo y elegir automáticamente el de mayor confianza, con la opción de forzar uno concreto.
- **Gestión administrativa:** el administrador puede ampliar el catálogo de datos con nuevos pares síntoma–especialidad, ajustar el umbral de confianza y los intentos de reintento, revisar los casos pendientes en el *log* de fallback y ejecutar el reentrenamiento de los modelos.
- **Ejecución local y privacidad por diseño:** todo (datos, modelos y predicción) se ejecuta en el equipo del usuario. Sin servicios cloud y sin modelos de IA propietarios de terceros.
- **Modularidad:** arquitectura por módulos para facilitar el mantenimiento y dejar la puerta abierta a nuevos canales de entrada o modelos adicionales.

B.3. Catálogo de requisitos

Los requisitos se agrupan en dos bloques: funcionales y no funcionales.

Requisitos funcionales

- **RF-1 Gestión de datos:** la aplicación debe gestionar el conjunto de datos de entrenamiento de forma persistente.
  - **RF-1.1 Carga del dataset principal:** el sistema debe cargar un CSV de partida con cuatro columnas (identificador de registro, descripción de síntomas, enfermedad asociada y especialidad médica), codificado en Latin-1 y separado por punto y coma.
  - **RF-1.2 Validación del dataset:** antes del entrenamiento, el sistema descarta valores nulos, líneas malformadas y registros incompletos.

  - **RF-1.3 Mantenimiento de un CSV de revisados:** las aportaciones generadas a partir del *log* de baja confianza se almacenan en un CSV adicional, independiente del dataset principal.
  - **RF-1.4 Incorporación directa al dataset principal:** el administrador debe poder añadir nuevos pares síntoma–especialidad directamente al dataset principal con un identificador correlativo único.
  - **RF-1.5 Combinación de fuentes para entrenamiento:** en el reentrenamiento, el sistema debe combinar el dataset principal y el CSV de revisados como conjunto de datos único.

- **RF-2 Preprocesado del texto:** el sistema aplica el mismo pipeline de PLN en entrenamiento y en predicción.
  - **RF-2.1 Limpieza y normalización:** el sistema debe convertir el texto a minúsculas, eliminar caracteres no alfabéticos preservando tildes y la letra ñ, y tokenizar el resultado.
  - **RF-2.2 Eliminación de palabras vacías:** se deben eliminar las *stopwords* del español antes de la vectorización.
  - **RF-2.3 Reducción morfológica (Stemming):** se debe aplicar el algoritmo Snowball para español sobre los *tokens* resultantes.
  - **RF-2.4 Generación de n-gramas:** la vectorización debe contemplar al menos unigramas y bigramas para captar expresiones clínicas compuestas como “dolor torácico” o “falta de aire”.
  - **RF-2.5 Coherencia entrenamiento/predicción:** el preprocesado aplicado al texto introducido por el paciente debe ser el mismo que el utilizado durante el entrenamiento.

- **RF-3 Modelos de clasificación:** la aplicación integra los cuatro modelos del proyecto, con una interfaz común para entrenarlos, persistirlos y consultarlos.
  - **RF-3.1 Modelo TF-IDF:** el sistema debe disponer de un clasificador basado en representación TF-IDF combinado con regresión logística como línea base.
  - **RF-3.2 Modelo TF-IDF + SVD:** el sistema debe disponer de un modelo que aplique reducción de dimensionalidad mediante TruncatedSVD sobre la representación TF-IDF antes de la clasificación.

- **RF-3.3 Modelo SVM:** el sistema debe disponer de un clasificador SVM con kernel lineal y `probability=True` entrenado sobre la representación TF-IDF.
- **RF-3.4 Modelo RNA (MLP):** el sistema debe disponer de un Perceptrón Multicapa entrenado con early stopping y regularización L2 (penalización proporcional al cuadrado de los pesos de la red) para evitar el sobreajuste.
- **RF-3.5 Persistencia de modelos:** los modelos entrenados deben serializarse en disco con `joblib` para evitar el reentrenamiento en cada arranque.
- **RF-3.6 Estimación de probabilidades:** cada modelo debe devolver, junto con la especialidad predicha, una puntuación de confianza expresada como porcentaje.

- **RF-4 Pestaña del paciente:** la aplicación debe ofrecer una pestaña destinada al paciente que permita la introducción de síntomas y la consulta de la especialidad asignada.
  - **RF-4.1 Entrada de síntomas por texto:** el paciente debe poder introducir sus síntomas escribiéndolos en un campo de texto libre.
  - **RF-4.2 Modo multimodelo (por defecto):** por defecto, el sistema debe ejecutar los cuatro modelos en paralelo y seleccionar automáticamente la predicción del modelo con mayor confianza.
  - **RF-4.3 Selección manual de modelo:** el paciente debe poder elegir alternativamente un modelo concreto entre TF-IDF, SVD, SVM o RNA.
  - **RF-4.4 Predicción de especialidad:** el sistema debe devolver la especialidad médica recomendada junto con la confianza asociada.
  - **RF-4.5 Visualización comparativa:** cuando se utilice el modo multimodelo, el sistema debe mostrar al paciente una tabla con los resultados de los cuatro modelos, identificando el modelo elegido.
  - **RF-4.6 Umbral de confianza configurable:** debe existir un umbral configurable por debajo del cual la predicción se considere dudosa.
  - **RF-4.7 Intentos extra configurables:** debe poderse configurar el número de intentos en los que se pide al paciente ampliar la descripción de los síntomas cuando la confianza es insuficiente.

- **RF-4.8 Solicitud de información adicional:** si la confianza queda por debajo del umbral y quedan intentos disponibles, el sistema debe pedir al paciente que amplíe la descripción y volver a evaluar la predicción.
- **RF-4.9 fallback por baja confianza:** si tras agotar los intentos la confianza sigue sin alcanzar el umbral, el sistema debe ofrecer al paciente dos opciones: aceptar la sugerencia del modelo o elegir manualmente otra especialidad de las disponibles en el dataset (con Medicina General como opción por defecto).
- **RF-4.10 Reporte de error sobre predicción aceptada:** aun cuando la confianza haya superado el umbral, el paciente debe poder marcar el resultado como incorrecto e indicar la especialidad correcta.
- **RF-4.11 Registro en log de baja confianza y reportes:** todos los casos resueltos por fallback y todos los reportes de error deben quedar registrados en un *log*, indicando síntomas y especialidad sugerida por el modelo o elegida por el paciente.

- **RF-5 Pestaña del administrador:** la aplicación debe ofrecer una pestaña destinada al administrador con las herramientas necesarias para mantener el sistema.
  - **RF-5.1 Autenticación del administrador:** el acceso a la pestaña debe requerir usuario y contraseña; la contraseña debe poder configurarse mediante variable de entorno o fichero de secretos, sin quedar incrustada en el código.
  - **RF-5.2 Configuración del triaje:** el administrador debe poder modificar el umbral mínimo de confianza y el número de intentos extra, persistiendo los cambios en un fichero de configuración (`config.json`).
  - **RF-5.3 Añadir síntomas y especialidad al dataset principal:** el administrador debe poder incorporar nuevas entradas (síntomas, enfermedad opcional, especialidad) al dataset principal.
  - **RF-5.4 Selección de especialidad existente o nueva:** la incorporación de nuevos registros debe permitir seleccionar una especialidad ya presente en el dataset o introducir una nueva.
  - **RF-5.5 Visualización del log:** el administrador debe poder consultar el listado de casos registrados en el *log* de baja confianza y reportes de error.

- **RF-5.6 Resolución de casos del log:** el administrador debe poder asignar manualmente la especialidad correcta a cada caso del *log* y volcarlo al CSV de revisados.
- **RF-5.7 Regeneración conjunta o individual de modelos:** el administrador debe poder regenerar los cuatro modelos a la vez a partir del dataset actualizado mediante una única acción o regenerarlos por separado.

Requisitos no funcionales

- **RNF-1 Ejecución standalone:** la aplicación debe ejecutarse de forma totalmente local, sin depender de APIs externas, servicios en la nube ni conexión a internet para la operativa principal.
- **RNF-2 Sin dependencia de IA propiedad de terceros:** el sistema no debe apoyarse en LLMs comerciales ni en servicios externos de clasificación clínica.
- **RNF-3 Privacidad por diseño:** los datos clínicos manipulados (síntomas, log de incidencias, dataset) deben permanecer en el equipo en el que se ejecuta la aplicación.
- **RNF-4 Ejecución sobre CPU:** los modelos deben ser lo bastante ligeros como para entrenarse y ejecutarse en un equipo modesto.
- **RNF-5 Rendimiento razonable:** los tiempos de respuesta de predicción y reentrenamiento deben de completarse en un plazo asumible.
- **RNF-6 Usabilidad:** la interfaz debe ser intuitiva, con una curva de aprendizaje baja.
- **RNF-7 Modularidad:** el código debe estar separado en módulos con responsabilidades bien delimitadas (carga de datos, preprocesado, vectorización, modelos, evaluación e interfaz).
- **RNF-8 Portabilidad:** el sistema, desarrollado en Python con dependencias acotadas en `requirements.txt`, debe poder ejecutarse en distintos sistemas operativos sin modificaciones sustanciales.

B.4. Especificación de requisitos

Esta sección recoge los actores del sistema, el diagrama general de casos de uso y la descripción de cada caso.

Actores del sistema

A01	**Paciente**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Descripción	Persona que utiliza la aplicación para describir sus síntomas y obtener una recomendación de especialidad médica.
Tipo	Usuario
Objetivo	Obtener una clasificación fiable de su sintomatología en una especialidad médica para orientar la solicitud de cita.
Responsabilidades	▪ Introducir sus síntomas en texto libre. ▪ Elegir el modo de clasificación (multimodelo o modelo concreto). ▪ Ampliar la descripción o elegir entre las alternativas propuestas si el sistema lo solicita. ▪ Reportar como incorrecta una predicción si lo considera oportuno.
Relaciones con casos de uso	CU-01, CU-02, CU-03, CU-04

Tabla B.1: A01 - Paciente.

A02	**Administrador**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Descripción	Usuario con privilegios que mantiene el catálogo de datos, la configuración del triaje y los modelos. Necesita autenticarse para acceder a la pestaña restringida.
Tipo	Usuario
Objetivo	Mantener actualizados los datos y los modelos, y ajustar el flujo de triaje cuando sea necesario.
Responsabilidades	▪ Configurar el umbral de confianza y los intentos extra. ▪ Añadir nuevos pares síntoma–especialidad al dataset principal. ▪ Revisar el *log* y resolver casos pendientes. ▪ Regenerar los modelos, de forma conjunta o individual.
Relaciones con casos de uso	CU-05, CU-06, CU-07, CU-08, CU-09

Tabla B.2: A02 - Administrador.

A03	**Motor de clasificación**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Descripción	Módulo interno que aplica el preprocesado y los cuatro modelos (TF-IDF, TF-IDF+SVD, SVM y RNA) sobre el texto introducido, y devuelve la especialidad predicha con su confianza. Puede usarse en modo individual o en modo multimodelo.
Tipo	Sistema
Objetivo	Clasificar texto libre en una especialidad médica y devolver una confianza asociada.
Responsabilidades	▪ Preprocesar el texto introducido. ▪ Aplicar uno o varios modelos de clasificación. ▪ Calcular el indicador de confianza por modelo. ▪ Seleccionar el modelo con mayor confianza en modo multimodelo.
Relaciones con casos de uso	CU-02, CU-03, CU-09

Tabla B.3: A03 - Motor de clasificación.

A04	**Almacenamiento local**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Descripción	Persistencia local del sistema: dataset principal y CSV de revisados, *log* de baja confianza y reportes de error, modelos serializados con `joblib` y fichero de configuración `config.json`.
Tipo	Sistema
Objetivo	Conservar datos, modelos y configuración en local, sin recurrir a servicios externos.
Responsabilidades	▪ Almacenar el dataset principal y el CSV de revisados. ▪ Registrar los casos en el *log*. ▪ Persistir los modelos entrenados y la configuración del triaje.
Relaciones con casos de uso	CU-03, CU-04, CU-05, CU-06, CU-07, CU-08, CU-09

Tabla B.4: A04 - Almacenamiento local.

Diagrama general de casos de uso

A continuación se muestran dos diagramas separados para mejorar la legibilidad: uno con los casos de uso del paciente y otro con los del administrador.

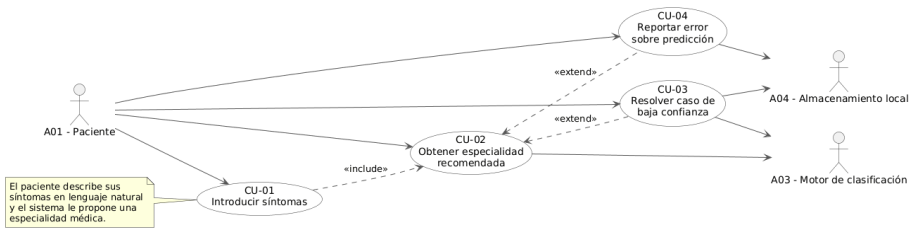

Figura B.1: Diagrama de casos de uso del paciente.

Casos de uso

CU-01	**Introducir síntomas**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A01 Paciente
Requisitos asociados	RF-4.1, RF-4.2, RF-4.3
Descripción	El paciente introduce sus síntomas en el formulario y selecciona el modo de clasificación (multimodelo o modelo concreto).
Precondición	La aplicación está abierta en la pestaña del paciente.
Acciones	1. El paciente accede a la pestaña Triaje. 2. Escribe sus síntomas en el campo de texto. 3. Selecciona el modelo (Todos por defecto, o uno concreto). 4. Pulsa el botón “Realizar triaje”.
Postcondición	El sistema dispone del texto y la configuración necesaria para iniciar la clasificación.
Excepciones	Campo de síntomas vacío al pulsar el botón.
Importancia	Alta

Tabla B.5: CU-01 Introducir síntomas.

CU-02	**Obtener especialidad recomendada**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A01 Paciente, A03 Motor de clasificación, A04 Almacenamiento local
Requisitos asociados	RF-2, RF-3, RF-4.2, RF-4.3, RF-4.4, RF-4.5
Descripción	El sistema procesa los síntomas y devuelve la especialidad recomendada y su porcentaje de confianza. En modo multimodelo, muestra además los resultados de los cuatro modelos.
Precondición	Existen modelos entrenados y persistidos y el paciente ha introducido síntomas.
Acciones	1. El sistema aplica el preprocesado sobre el texto introducido. 2. Ejecuta los cuatro modelos (modo multimodelo) o el modelo seleccionado. 3. En modo multimodelo, escoge la predicción con mayor confianza. 4. Muestra al paciente la especialidad recomendada, la confianza y la tabla comparativa cuando proceda.
Postcondición	El paciente conoce la especialidad recomendada y, en su caso, el desempeño de cada modelo.
Excepciones	Modelos no disponibles o corruptos; ningún modelo devuelve un resultado válido.
Importancia	Alta

Tabla B.6: CU-02 Obtener especialidad recomendada.

CU-03	**Resolver caso de baja confianza**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A01 Paciente, A03 Motor de clasificación, A04 Almacenamiento local
Requisitos asociados	RF-4.6, RF-4.7, RF-4.8, RF-4.9, RF-4.11
Descripción	Si la confianza está por debajo del umbral, el sistema pide más síntomas durante los intentos configurados. Si tras agotarlos la confianza sigue siendo insuficiente, el paciente puede aceptar la sugerencia del modelo o elegir manualmente otra especialidad. El caso queda registrado en el *log*.
Precondición	Se ha realizado una primera predicción cuya confianza es inferior al umbral configurado.
Acciones	1. El sistema solicita al paciente ampliar la descripción de los síntomas. 2. El paciente añade información complementaria. 3. El sistema reevalúa la predicción con el mismo modelo. 4. Si la confianza es suficiente, devuelve la especialidad y termina. 5. Si la confianza sigue siendo insuficiente y quedan intentos, se repite el paso 1. 6. Agotados los intentos, el sistema ofrece al paciente aceptar la sugerencia del modelo o elegir manualmente otra especialidad (Medicina General por defecto). 7. El caso queda registrado en el *log* con motivo `baja_confianza`.
Postcondición	El paciente recibe una recomendación final y el caso queda registrado en el *log*.
Excepciones	El paciente cancela el proceso antes de finalizar los intentos.
Importancia	Alta

Tabla B.7: CU-03 Resolver caso de baja confianza.

CU-04	**Reportar error sobre predicción aceptada**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A01 Paciente, A04 Almacenamiento local
Requisitos asociados	RF-4.10, RF-4.11
Descripción	Cuando la confianza ha superado el umbral, el paciente puede marcar la predicción como incorrecta e indicar la especialidad real.
Precondición	El paciente ha recibido una recomendación de especialidad con confianza por encima del umbral.
Acciones	1. El paciente pulsa el botón "Reportar error". 2. Selecciona del catálogo de especialidades la que considera correcta. 3. Confirma el envío del reporte. 4. El sistema registra el caso en el *log* con motivo `reporte_error`.
Postcondición	El caso queda almacenado en el *log* para su revisión posterior por el administrador.
Excepciones	Error de escritura en el *log*; cancelación por parte del paciente.
Importancia	Media

Tabla B.8: CU-04 Reportar error sobre predicción aceptada.

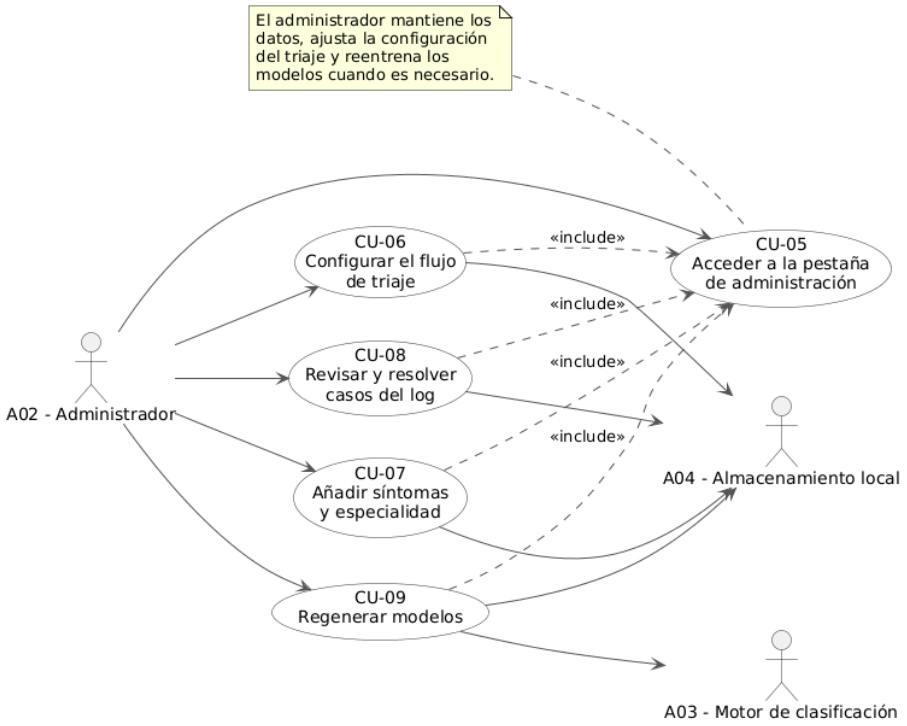

Figura B.2: Diagrama de casos de uso del administrador.

CU-05	**Acceder a la pestaña de administración**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A02 Administrador
Requisitos asociados	RF-5.1
Descripción	El administrador inicia sesión con usuario y contraseña para acceder a las funcionalidades restringidas.
Precondición	Existe una contraseña configurada en `st.secrets`, en variable de entorno o en fichero de secretos.
Acciones	1. El administrador selecciona la pestaña Administración. 2. Introduce usuario y contraseña. 3. Pulsa el botón “Entrar”. 4. El sistema valida las credenciales mediante comparación segura.
Postcondición	El administrador queda autenticado y accede a las herramientas de gestión.
Excepciones	Credenciales incorrectas; no hay contraseña configurada.
Importancia	Alta

Tabla B.9: CU-05 Acceder a la pestaña de administración.

CU-06	**Configurar el flujo de triaje**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A02 Administrador, A04 Almacenamiento local
Requisitos asociados	RF-4.6, RF-4.7, RF-5.2
Descripción	El administrador ajusta el umbral de confianza y el número de intentos extra de ampliación.
Precondición	El administrador está autenticado.
Acciones	1. El administrador modifica el umbral mediante un *slider*. 2. Modifica el número de intentos extra. 3. Pulsa "Guardar configuración". 4. El sistema persiste los valores en `config.json`.
Postcondición	Las próximas predicciones utilizan la nueva configuración.
Excepciones	Error de escritura en disco.
Importancia	Media

Tabla B.10: CU-06 Configurar el flujo de triaje.

CU-07	**Añadir síntomas y especialidad**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A02 Administrador, A04 Almacenamiento local
Requisitos asociados	RF-1.4, RF-5.3, RF-5.4
Descripción	El administrador añade nuevos pares síntoma–especialidad al dataset principal, eligiendo entre especialidades existentes o introduciendo una nueva.
Precondición	El administrador está autenticado.
Acciones	1. El administrador introduce la descripción de los síntomas. 2. Selecciona una especialidad existente o escribe una nueva. 3. Opcionalmente, indica la enfermedad asociada. 4. Pulsa “Guardar registro”. 5. El sistema asigna un identificador correlativo y añade la fila al dataset principal.
Postcondición	El dataset principal contiene una entrada nueva, disponible para el próximo reentrenamiento.
Excepciones	Datos incompletos; error de escritura.
Importancia	Alta

Tabla B.11: CU-07 Añadir síntomas y especialidad.

CU-08	**Revisar y resolver casos del log**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A02 Administrador, A04 Almacenamiento local
Requisitos asociados	RF-1.3, RF-5.5, RF-5.6
Descripción	El administrador consulta los casos del *log* (baja confianza y reportes de error). Para los que decida revisar, asigna la especialidad correcta y vuelca el caso al CSV de revisados.
Precondición	Existe al menos un caso registrado en el *log*.
Acciones	1. El administrador accede al apartado de log. 2. Visualiza cada caso con los síntomas. 3. Para cada caso a revisar, selecciona la especialidad adecuada. 4. Pulsa "Añadir a datos" para incorporar el caso al CSV de revisados.
Postcondición	El caso queda incorporado al CSV de revisados; el log mantiene su registro histórico.
Excepciones	Especialidad no válida; fallo en la escritura del CSV.
Importancia	Alta

Tabla B.12: CU-08 Revisar y resolver casos del log.

CU-09	**Regenerar modelos**
Versión	1.0
Autor	Ángel Ortiz de Lejarazu Sánchez
Actores	A02 Administrador, A03 Motor de clasificación, A04 Almacenamiento local
Requisitos asociados	RF-1.5, RF-3.5, RF-5.7
Descripción	El administrador reentrena los modelos a partir del dataset principal combinado con el CSV de revisados. Puede hacerlo de forma conjunta o regenerar un solo modelo.
Precondición	Existen los CSV de datos accesibles y los scripts de regeneración.
Acciones	1. El administrador pulsa "Regenerar todos los modelos" o un botón individual. 2. El sistema invoca el script correspondiente a cada modelo y muestra un indicador de progreso. 3. Cada script combina el dataset principal y el CSV de revisados. 4. El script entrena el modelo y persiste el resultado en disco. 5. El sistema notifica al administrador el resultado de cada regeneración.
Postcondición	Los modelos persistidos reflejan los datos actualizados, incluyendo las aportaciones del administrador.
Excepciones	Datos corruptos; error durante el entrenamiento; fallo de escritura; script no encontrado.
Importancia	Alta

Tabla B.13: CU-09 Regenerar modelos.

Apéndice C

Especificación de diseño

C.1. Introducción

Este anexo recoge las decisiones de diseño del prototipo de triaje clínico desarrollado en el TFG, organizadas en tres planos. El diseño de datos describe qué información maneja la aplicación y cómo se guarda en el sistema de ficheros. El diseño arquitectónico explica cómo se reparte el software en módulos y cómo `app.py` orquesta al resto. Y el diseño procedimental, ya con diagramas de secuencia, sigue los casos de uso del anexo B paso a paso.

Hay dos decisiones tomadas al inicio que merece la pena aclarar antes de entrar en detalle, porque condicionan todo lo demás. La primera fue prescindir de una base de datos relacional: por filosofía standalone y para evitar dependencias, los datos viven en un único archivo CSV en disco. La segunda fue integrar cuatro motores de clasificación distintos — TF-IDF, TF-IDF con reducción SVD, un SVM y una RNA de tipo MLP—, lo que obliga a un diseño donde cada modelo tiene su propio módulo y la aplicación puede invocarlos de forma uniforme.

C.2. Diseño de datos

El prototipo no usa una base de datos. Trabaja sobre un fichero plano que se carga en memoria al arrancar. Para un corpus de unos miles de filas, estable y de un solo usuario, un CSV es suficiente y mucho más fácil de versionar e inspeccionar que un motor SQL.

Entidades del sistema

Aunque no haya un esquema relacional formal, sí se pueden distinguir tres entidades lógicas que estructuran la información:

- **RegistroSintoma**: una fila del dataset principal. Contiene el identificador (*record_id*), el texto libre con los síntomas (*symptoms_text*), la enfermedad asociada cuando se conoce (*disease*, opcional) y la especialidad médica de destino (*specialty*), que es la clase objetivo del clasificador.

- **PrediccionTriaje**: el resultado de una consulta del paciente. Vive solo en memoria, no se guarda. Lleva la *especialidad* devuelta, la *confianza* asociada y el *modelo_origen* que la produjo (TF-IDF, SVD, SVM o RNA).

- **ModeloEntrenado**: la versión serializada de un clasificador con todo lo que necesita para volver a predecir tras un reinicio: el vectorizador, el reductor de dimensionalidad (cuando aplica), el clasificador y la codificación de etiquetas. Se guarda en uno o varios ficheros `.joblib` dentro de la carpeta del modelo correspondiente, y se carga con la misma librería `joblib` al invocar la función de predicción.

Organización del almacenamiento

La persistencia se reparte en un CSV de datos y varias carpetas que contienen tanto el código de cada modelo como los ficheros `.joblib` con el modelo entrenado. La tabla C.1 lo resume.

Ruta	**Uso**
`Datos/Dataset.5000.Registros.csv`	Dataset principal con 5 000 registros etiquetados; el nombre completo incluye la fecha de generación.
`TF_IDF/`	Módulo del clasificador TF-IDF y ficheros `.joblib` con el modelo entrenado.
`SVD/`	Módulo del clasificador con reducción SVD y sus ficheros `.joblib`.
`SVM/`	Módulo del clasificador SVM y sus ficheros `.joblib`.
`RNA/`	Módulo de la red neuronal y sus ficheros `.joblib`.
`.streamlit/secrets.toml`	Contraseña del administrador, bajo la clave `admin_password`.

Tabla C.1: Ficheros y carpetas con datos persistentes

El dataset usa `;` como separador y `latin1` como codificación. La elección de `latin1` no fue caprichosa: el UTF-8 daba problemas con los acentos en algunas de las herramientas de edición con las que probé el fichero, y `latin1` resolvió el tema sin más. La lectura se hace con `pandas` y se cachea en sesión con `@st.cache_data`, de modo que el coste de E/S solo se paga al arrancar.

Relaciones entre entidades

La figura C.1 resume las relaciones entre las entidades anteriores. El dataset alimenta el entrenamiento de los cuatro modelos. Cada modelo produce sus propios ficheros `.joblib`. Y, cuando el administrador añade nuevos registros, el ciclo se reinicia con un reentrenamiento que vuelve a leer el CSV y reescribe esos ficheros.

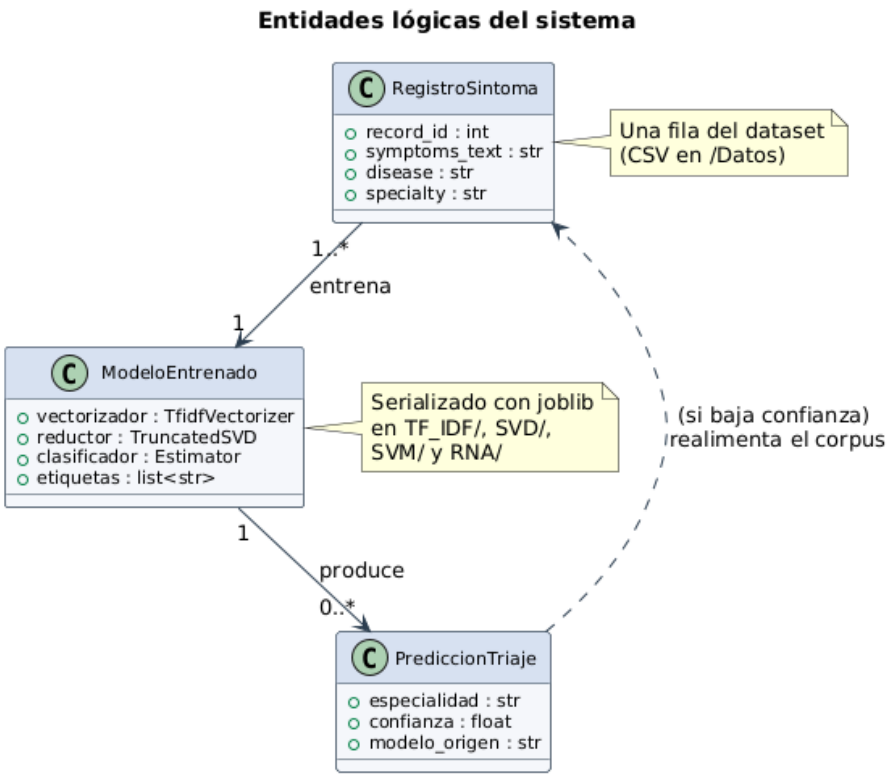

Figura C.1: Relaciones lógicas entre entidades del sistema

0.85

C.3. Diseño arquitectónico

Visión general por módulos

La aplicación gira alrededor de `app.py`, que es el punto de entrada de Streamlit y el que orquesta al resto. A su alrededor hay cuatro módulos independientes, uno por cada modelo, y una capa de datos compartida. La figura C.2 lo dibuja.

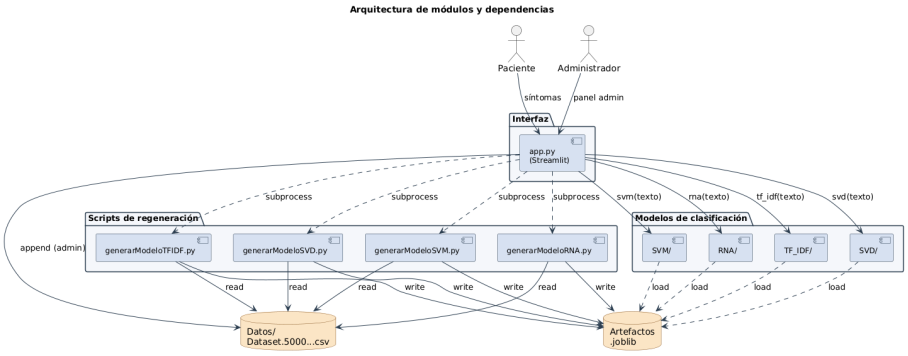

Figura C.2: Diagrama arquitectónico de módulos y dependencias

0.9

Las responsabilidades se reparten así:

- **Interfaz (`app.py`)**: tiene las dos páginas de la aplicación Streamlit —triaje y administración—, que se conmutan desde la barra lateral con un `st.sidebar.radio`. Aquí también vive el formulario de *login* del administrador, la captura de síntomas por voz con `streamlit-mic-recorder` y la presentación de los resultados.
- **Módulos de clasificación** (`TF_IDF/`, `SVD/`, `SVM/`, `RNA/`): cada uno expone una función pública con el mismo nombre que el módulo (`tf_idf`, `svd`, `svm`, `rna`). La función recibe un texto y devuelve la tupla `(especialidad, confianza)`. Por dentro carga el modelo serializado con `joblib` (vectorizador TF-IDF, transformación SVD cuando aplica y clasificador entrenado), preprocesa el texto y predice.
- **Scripts de regeneración**: cada carpeta de modelo lleva además un script `generarModelo<NOMBRE>.py` que se ejecuta por separado, lee el dataset, entrena el modelo y lo vuelve a guardar en disco con `joblib`. `app.py` los lanza con `subprocess` cuando el administrador pulsa el botón correspondiente.
- **Datos compartidos** (`Datos/`): el corpus, en un único sitio. Lo leen los módulos de clasificación y los scripts de regeneración, y la pestaña de administración añade filas al mismo fichero.

Una aclaración sobre los modelos. Las cuatro funciones de predicción comparten firma, pero el proyecto no usa una clase abstracta común ni un

patrón *Strategy* formal. `app.py` importa las cuatro funciones y elige entre ellas con un `if/elif` sobre el valor del `selectbox`.

Lo hice así a propósito. Mantiene el orquestador legible y deja a cada módulo libertad para implementar su preprocesado y su política de confianza como mejor le venga. El precio es claro: añadir un quinto modelo obliga a tocar `app.py` y el listado del `selectbox`, en lugar de bastar con registrar una clase nueva.

Estructura de directorios

Trasladado a la estructura real del repositorio:

- `app.py`: punto de entrada de Streamlit y orquestador.
- `TF_IDF/`: módulo del clasificador TF-IDF, con la función `tf_idf()` y `generarModeloTFIDF.py`.
- `SVD/`: variante con reducción SVD y `generarModeloSVD.py`.
- `SVM/`: clasificador SVM y `generarModeloSVM.py`.
- `RNA/`: red neuronal y `generarModeloRNA.py`.
- `Datos/`: dataset principal y datasets auxiliares cuando los hay.
- `.streamlit/`: configuración de Streamlit y la contraseña de administración.
- `scripts/`: utilidades sueltas para preparar y verificar datos.
- `docs/`: documentación complementaria del proyecto.
- `vozATexto1.py` y `vozATexto2.py`: pruebas iniciales del componente de voz, conservadas como referencia.
- `requirements.txt`: dependencias del proyecto.

Autenticación de la pestaña de administración

La pestaña de administración está protegida por un mecanismo de autenticación sencillo, todo él dentro de `app.py`. La contraseña esperada se busca, por este orden, en `st.secrets`, después en un fichero `secrets.toml` dentro de `.streamlit/` y, si no aparece en ninguno de los dos, en la variable de entorno `TRIAJE_ADMIN_PASSWORD`. La comparación usa `hmac.compare_digest`

en vez de un `==` normal, para no ser vulnerable a ataques por tiempo. El estado de sesión se guarda en `st.session_state.admin_autenticado` y se puede cerrar desde un botón *Cerrar sesión* en la barra lateral.

Arquitectura de despliegue

El despliegue es muy simple: un único proceso Python ejecuta Streamlit, que sirve la interfaz en un puerto local (por defecto el 8501). Los modelos se cargan cuando hace falta y la aplicación trabaja contra el CSV alojado en una carpeta del propio equipo. Sin servicios externos, sin conexiones de red salientes, sin nada en la nube: los datos del paciente no salen de la máquina, que es la idea standalone planteada en la memoria y la condición no funcional del anexo B.

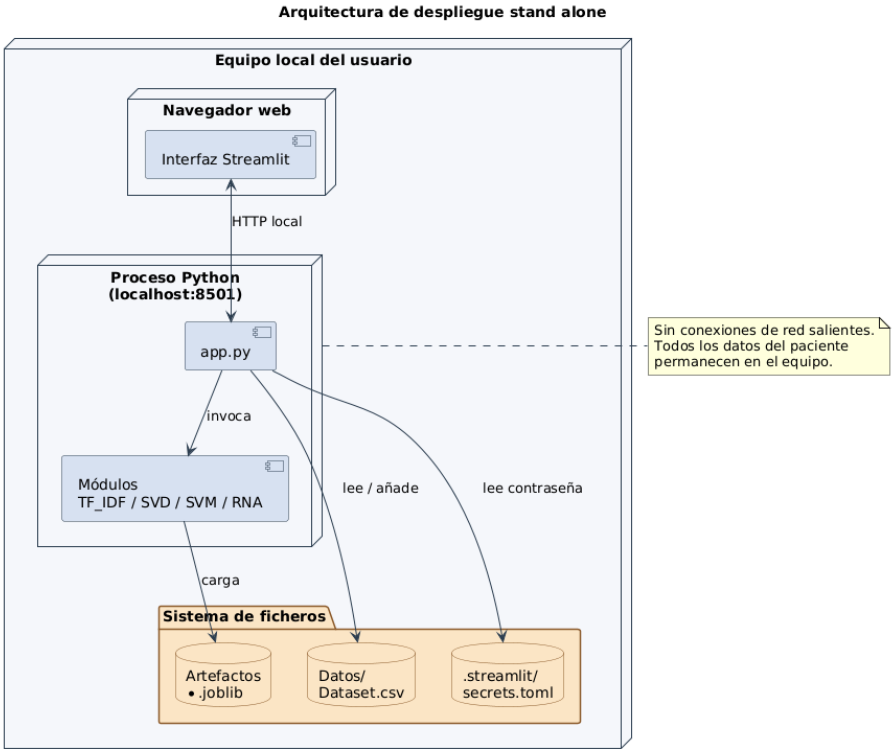

Figura C.3: Arquitectura de despliegue del prototipo

0.7

C.4. Diseño procedimental

Los apartados que siguen describen los flujos más representativos con diagramas de secuencia, cada uno asociado a su caso de uso del anexo B.

Introducir síntomas y obtener especialidad (CU-01, CU-02)

Es el flujo principal. El paciente teclea sus síntomas en la pestaña de triaje, elige el motor de IA y obtiene una especialidad con su porcentaje de confianza. Por defecto la entrada es por teclado; la interfaz también ofrece un botón de dictado para transcribir por voz. La secuencia completa está en la figura C.4.

Los pasos son estos:

1. El paciente entra en la página de triaje y teclea los síntomas. Si lo prefiere, puede pulsar el botón de dictado: `streamlit-mic-recorder` graba en el navegador, transcribe y vuelca el texto en el mismo campo, donde el paciente todavía puede corregirlo antes de seguir.
2. Revisa el texto, elige el motor en el `selectbox` (TF-IDF, SVD, SVM o RNA) y pulsa el botón de triaje.
3. `app.py` comprueba que el campo no esté vacío y, según la opción elegida, invoca la función del módulo correspondiente.
4. El módulo preprocesa el texto, predice y devuelve la tupla con la especialidad y la confianza.
5. `app.py` construye el mensaje de salida y lo muestra al paciente con `st.success` y `st.info`.

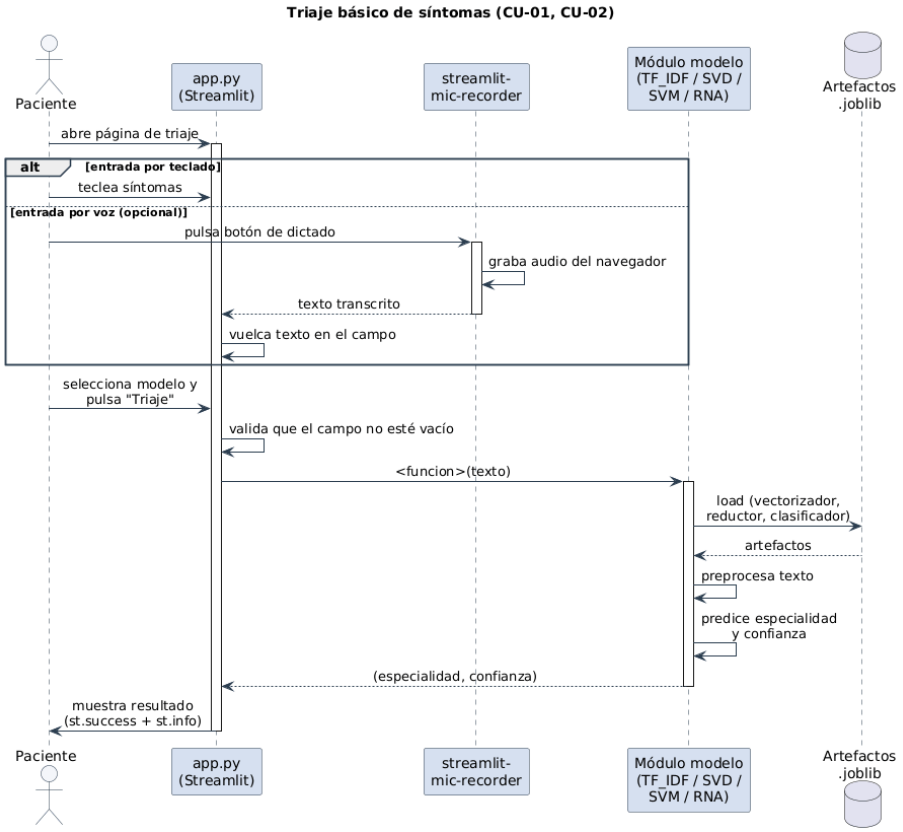

Figura C.4: Diagrama de secuencia - Triaje básico de síntomas

0.95

Un matiz sobre el alcance actual: `app.py` muestra el porcentaje de confianza, pero el código revisado todavía no aplica un umbral de derivación ni encadena una segunda ronda de preguntas. La ampliación de síntomas y la derivación automática a atención general aparecen como objetivo en la memoria, pero por ahora son evolución prevista; cuando entren en producción se documentarán aquí.

Añadir nuevos síntomas desde administración (CU-04)

La pestaña de administración deja al usuario autenticado incorporar nuevos pares de síntomas y especialidad al corpus. El flujo arranca con un formulario que tiene un campo de texto libre para los síntomas, dos vías para fijar la especialidad —elegirla del `selectbox` de las ya existentes o teclear una nueva— y un campo opcional para la enfermedad asociada.

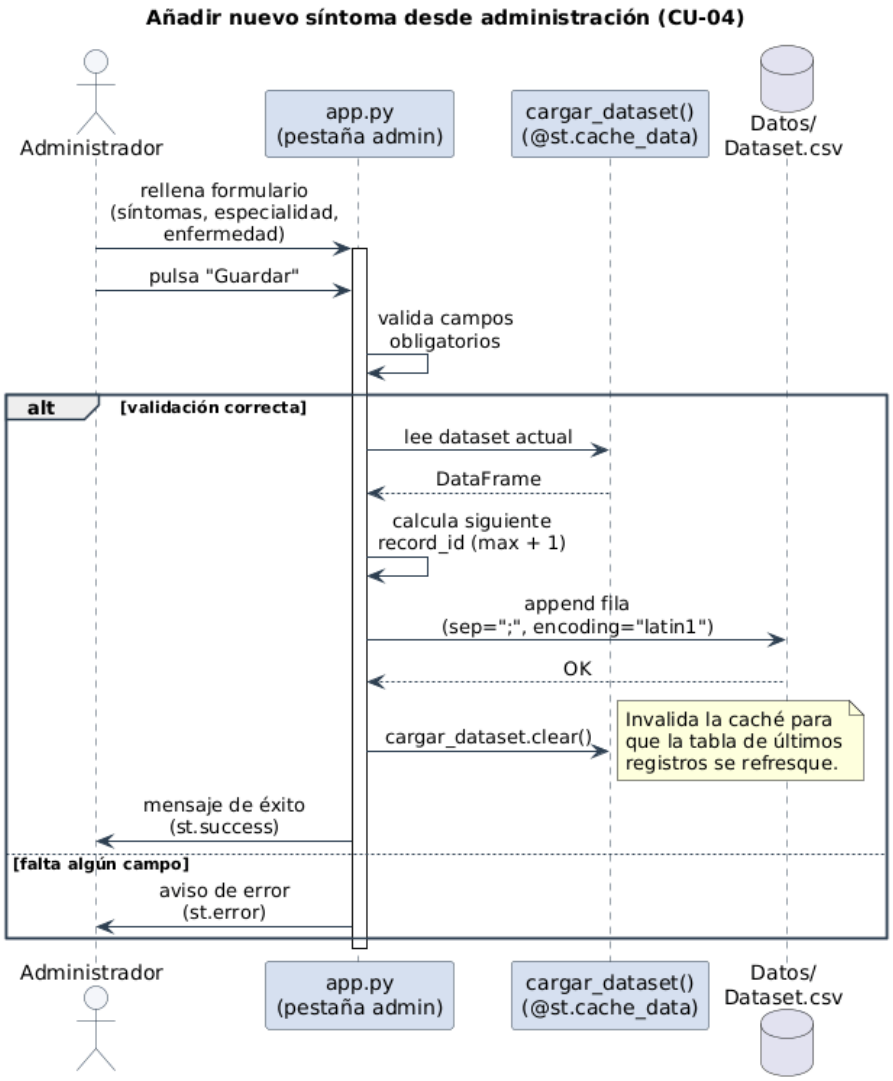

Figura C.5: Diagrama de secuencia - Añadir síntoma desde administración

0.95

Al pulsar Guardar, `app.py` valida los campos obligatorios, calcula el siguiente identificador a partir del máximo actual de `record_id` y anexa la fila al CSV usando el módulo `csv` estándar, respetando el separador y la codificación originales. A continuación invalida la caché de `cargar_dataset` con `cargar_dataset.clear()`, para que la tabla de últimos registros que se ve al final de la pestaña se refresque al instante. Los modelos en memoria no se enteran de este cambio; solo lo notarán cuando se reentrene explícitamente.

Regeneración de modelos (CU-06)

Esta es la operación más costosa de la aplicación: vuelve a entrenar cada modelo sobre el corpus actualizado. La pestaña de administración tiene cuatro botones independientes, uno por modelo, repartidos en dos columnas. Cada botón lanza el script de regeneración correspondiente como subproceso del intérprete Python actual.

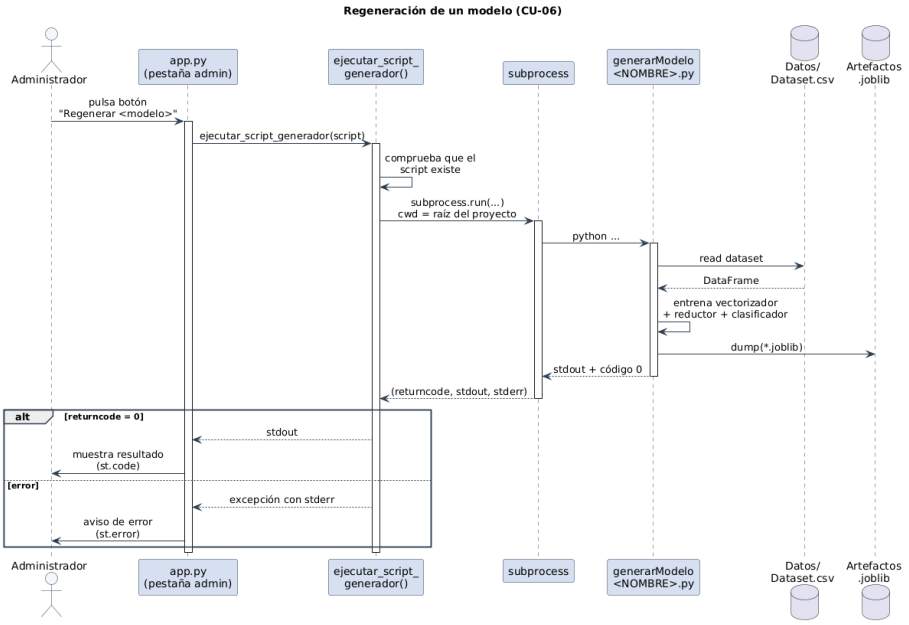

Figura C.6: Diagrama de secuencia - Regeneración de un modelo

0.95

Todo el proceso vive en la función `ejecutar_script_generador`, que hace lo siguiente:

1. Comprueba que el script existe en la ruta esperada (por ejemplo, `TF_IDF/generarModeloTFIDF.py`).
2. Lo lanza con `subprocess.run`, fijando `cwd` en la raíz del proyecto para que las rutas relativas dentro del script funcionen.
3. Captura la salida estándar y la de error; si el código de retorno no es cero, lanza una excepción con el contenido capturado.
4. Si todo va bien, devuelve la salida estándar para que `app.py` la pinte en un bloque `st.code`.

Sacar el entrenamiento a un subproceso tiene un par de ventajas prácticas. Por un lado, aísla la memoria: el modelo recién entrenado no convive con la aplicación en el mismo intérprete, así que no hay choques de estado ni de bibliotecas. Por otro, si un entrenamiento se cuelga puedo matar el subproceso sin tumbar la aplicación. Y como bonus, los mismos scripts se pueden ejecutar desde la línea de comandos, fuera de Streamlit, lo que viene bien durante el desarrollo y las validaciones.

Acceso autenticado al panel de administración (CU-07)

La autenticación se ejecuta antes de pintar nada del panel de administración. Cuando el usuario elige `Administración` en la barra lateral, `app.py` mira `st.session_state.admin_autenticado`; si la sesión todavía no está autenticada, sale el formulario con usuario y contraseña.

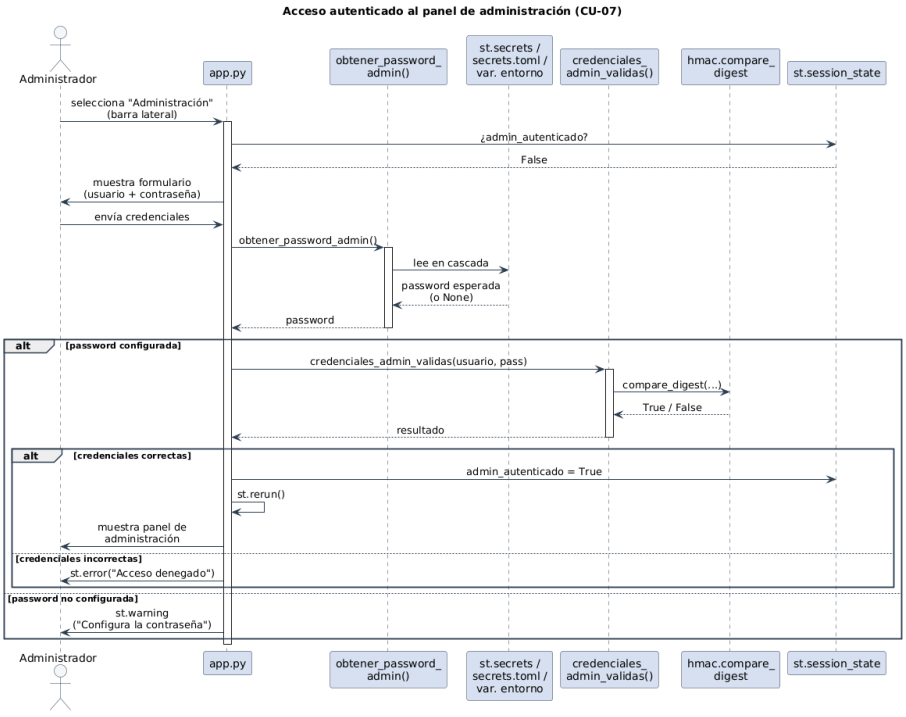

Figura C.7: Diagrama de secuencia - Acceso al panel de administración

0.95

Al enviar el formulario, la función `credenciales_admin_validas` compara con `hmac.compare_digest`. Si coincide, marca la sesión como autenticada

y llama a `st.rerun` para repintar la página, ya con el contenido administrativo. Si no, muestra error y el formulario sigue visible. Y si la contraseña no está configurada en ningún sitio, la aplicación avisa expresamente, para que un despliegue mal configurado no deje el panel abierto.

C.5. Resumen del stack tecnológico

La tabla C.2 resume las tecnologías que hay detrás de cada parte del prototipo, para emparejar cada bloque del diseño con la librería o herramienta concreta que lo implementa.

Componente	**Tecnología**
Interfaz de usuario	Streamlit (`app.py`)
Preprocesado y NLP	TF-IDF (`TfidfVectorizer` de scikit-learn)
Reducción de dimensionalidad	TruncatedSVD (scikit-learn)
Modelos de clasificación	Logistic Regression, SVM y MLP (scikit-learn)
Serialización de modelos	`joblib`
Manejo del dataset	`pandas` + módulo `csv` estándar
Persistencia	Archivos CSV en disco (carpeta `Datos/`)
Autenticación admin	`hmac.compare_digest` + `st.secrets`
Regeneración de modelos	`subprocess` sobre scripts `generarModelo<NOMBRE>.py`

Tabla C.2: Resumen del stack tecnológico del prototipo

La tabla reúne los componentes del flujo principal de clasificación. Aparte de ellos, la interfaz incluye un botón de dictado basado en `streamlit-mic-recorder` que sirve como vía de entrada alternativa al teclado. No le he dedicado fila propia porque su papel es secundario: traduce voz a texto y el resto del sistema sigue funcionando como si lo hubiera tecleado el paciente.

Apéndice D

Documentación técnica de programación

D.1. Introducción

Este apéndice está pensado para cualquier persona que quiera entender cómo está organizado el código del proyecto, ponerlo en marcha en su propio equipo y modificarlo. No es un manual de uso (eso queda para el apéndice de usuario), sino una guía técnica para programadores.

La aplicación es un prototipo de triaje médico escrito en Python que se ejecuta de forma local con Streamlit [7]. Recibe síntomas en lenguaje natural (escritos o dictados por voz) y devuelve la especialidad médica más adecuada, utilizando para ello cuatro modelos distintos: TF-IDF, TF-IDF combinado con SVD, SVM y una red neuronal de tipo MLP. Cada modelo se entrena por separado y se guarda en disco como un fichero `.joblib` mediante `joblib.dump`, de manera que la aplicación pueda cargarlo en memoria cuando lo necesite sin tener que volver a entrenar nada.

A lo largo del apéndice se explica cómo está repartido el código entre carpetas, qué herramientas hacen falta para montar el entorno, cómo se obtiene el proyecto desde GitHub, cómo se entrenan los modelos y se arranca la aplicación, y por último cómo se ha probado el sistema.

D.2. Estructura de directorios

El proyecto sigue una organización sencilla, con una carpeta por modelo y los datos y los logs separados del código. Esta separación facilita modificar

un modelo sin tocar el resto y mantener limpios los ficheros que genera la aplicación durante su uso.

- `/`: en la raíz se encuentra el fichero `app.py`, que es el punto de entrada de la aplicación Streamlit, el fichero `config.json` con la configuración del umbral de confianza y las rondas de ampliación, el `README.md` y el `.gitignore`.
- `/Datos`: contiene el conjunto de entrenamiento principal (`Dataset.5000.Registros.Marz.ID.Sintomas.Enfermedad.Especialidad.csv`) y el CSV ampliado (`triajes_baja_confianza_revisados.csv`) que se genera cuando el administrador revisa entradas del log y las añade al dataset.
- `/TF_IDF`: incluye el módulo `tf_idf.py`, con la función de inferencia, y el script `generarModeloTFIDF.py`, que entrena el modelo base de TF-IDF con Regresión Logística. El modelo entrenado se guarda como `tfidf_model.joblib` dentro de la misma carpeta.
- `/SVD`: contiene `SVD.py` y `generarModeloSVD.py`. El script de entrenamiento aplica TF-IDF seguido de TruncatedSVD con 100 componentes y, finalmente, un clasificador de Regresión Logística. El modelo resultante se almacena en `svd_model.joblib`.
- `/SVM`: contiene `SVM.py` y `generarModeloSVM.py`. El entrenamiento utiliza `Pipeline` de scikit-learn junto con `GridSearchCV` para ajustar el parámetro `C` y el tamaño del vocabulario. El modelo se guarda en `svm_model.joblib`.
- `/RNA`: contiene `RNA.py` y `generarModeloRNA.py`. El entrenamiento utiliza un MLP con *early stopping*, sobre el que también se aplica `GridSearchCV` para probar arquitecturas y funciones de activación. El modelo final queda en `rna_model.joblib`.
- `/logs`: directorio donde se almacena `triaje_fallback_log.csv`. Aquí se registran las consultas que el sistema no ha podido clasificar con suficiente confianza, junto con el modelo utilizado, los síntomas, la especialidad sugerida por el modelo y la especialidad finalmente elegida por el paciente.
- `/.streamlit`: contiene la configuración de Streamlit y el fichero de *secrets*, donde se guarda la contraseña del administrador. Este fichero está excluido del control de versiones.

- `/tex` y `/img`: contienen los ficheros LaTeX y las imágenes utilizados para la redacción de la memoria y los anexos.

D.3. Manual del programador

Herramientas necesarias

Para preparar un entorno de desarrollo equivalente al utilizado durante el TFG, hace falta instalar los siguientes elementos:

- **Python 3.11 o superior**. Es el lenguaje en el que está escrito todo el código.
- **Git**, para clonar el repositorio y mantener el control de versiones.
- **Visual Studio Code** (recomendado) u otro editor con soporte para Python.
- **Una cuenta de GitHub**, opcional, si se quiere contribuir al repositorio o utilizar *GitHub Copilot* como ayuda durante la programación.

Python

Toda la aplicación se ha desarrollado en Python 3.11. Se puede descargar desde python.org/downloads. Durante la instalación en Windows conviene marcar la opción *Add Python to PATH*, para poder ejecutar `python` y `pip` desde cualquier terminal.

Tras la instalación se puede comprobar la versión abriendo una terminal y ejecutando:

```
python --version
pip --version
```

Visual Studio Code

Como editor se ha utilizado Visual Studio Code. Una vez instalado, se recomienda añadir desde el panel de extensiones (`Ctrl+Shift+X`) las siguientes:

- **Python** (Microsoft): proporciona el intérprete, el depurador y la integración con entornos virtuales.

- **Pylance**: análisis estático y autocompletado.
- **GitLens**: facilita la consulta del historial de cambios sin salir del editor.
- **GitHub Copilot** (opcional): ayuda durante la escritura de código. Es gratuito para estudiantes a través del programa *GitHub Education*.

Git

Git puede descargarse desde git-scm.com. En Windows se recomienda dejar las opciones por defecto del asistente, ya que permite trabajar tanto con *Git Bash* como con la terminal de Windows.

Obtención del código fuente

El código se aloja en un repositorio público de GitHub [4]. Para descargarlo, basta con abrir una terminal y ejecutar:

```
git clone https://github.com/<usuario>/<repositorio>.git
cd <repositorio>
```

Tras el clonado se obtiene la estructura de directorios descrita en la sección anterior.

Entorno virtual y dependencias

Para evitar interferencias con otros proyectos de Python instalados en el equipo, se recomienda crear un entorno virtual antes de instalar las dependencias. Desde la raíz del proyecto:

```
python -m venv .venv
```

A continuación, se activa el entorno. En Windows, desde *PowerShell*:

```
.venv\Scripts\Activate.ps1
```

Y en Linux o macOS:

```
source .venv/bin/activate
```

Una vez activo el entorno (el prompt mostrará `(.venv)` al inicio), se instalan las librerías necesarias. Si el repositorio incluye un fichero `requirements.txt`, basta con:

```
pip install -r requirements.txt
```

Si no, las dependencias mínimas que utiliza el proyecto son las siguientes:

```
pip install streamlit pandas numpy scikit-learn nltk joblib
pip install streamlit-mic-recorder
```

La primera vez que se ejecute el código de los modelos, NLTK descargará automáticamente el corpus de palabras vacías en español:

```
import nltk
nltk.download('stopwords')
```

Esta descarga solo se realiza una vez por equipo y se gestiona internamente desde los propios módulos.

Configuración del acceso de administrador

La pestaña de administración está protegida por una contraseña que se lee desde un fichero de *secrets* de Streamlit o desde una variable de entorno. La aplicación busca, por este orden:

1. El valor `admin_password` en los *secrets* de Streamlit.
2. El fichero `.streamlit/secrets.toml.aos006`.
3. La variable de entorno `TRIAJE_ADMIN_PASSWORD`.

Para configurarlo de forma local, basta con crear el fichero `.streamlit/secrets.toml` en la raíz del proyecto con el siguiente contenido:

```
admin_password = "tu_contraseña"
```

Este fichero está excluido del repositorio mediante `.gitignore`, por lo que la contraseña nunca se sube a GitHub. El usuario de administración es `admin`.

D.4. Compilación, instalación y ejecución del proyecto

Al tratarse de una aplicación Python interpretada, no hay una fase de compilación propiamente dicha. El proceso para tener la aplicación en marcha consiste en preparar los datos, entrenar los modelos y arrancar la interfaz Streamlit.

Entrenamiento de los modelos

Antes de poder utilizar la aplicación, deben existir los cuatro ficheros `.joblib` con los modelos entrenados. Cada modelo tiene su propio script de entrenamiento, que se ejecuta de forma independiente. Desde la raíz del proyecto y con el entorno virtual activo:

```
python TF_IDF/generarModeloTFIDF.py
python SVD/generarModeloSVD.py
python SVM/generarModeloSVM.py
python RNA/generarModeloRNA.py
```

Los dos primeros scripts (TF-IDF y SVD) terminan en pocos segundos, ya que utilizan únicamente Regresión Logística sin búsqueda de hiperparámetros. Los scripts de SVM y de la red neuronal son más lentos, porque aplican GridSearchCV sobre validación cruzada estratificada con cinco particiones. En un equipo modesto, el entrenamiento de la red neuronal puede tardar varios minutos.

Si solo se quieren regenerar los modelos sin entrar a línea de comandos, la propia aplicación incluye un botón por modelo dentro de la pestaña de administración. Este botón ejecuta internamente el script correspondiente mediante `subprocess.run` y muestra la salida en la interfaz si la regeneración tiene éxito.

Cada script de entrenamiento sigue el mismo esquema:

1. Carga el CSV de entrenamiento con `pandas.read_csv`.
2. Aplica el preprocesado de texto (paso a minúsculas, eliminación de caracteres no alfabéticos, eliminación de palabras vacías y Stemming con `SnowballStemmer`).

3. Entrena el modelo correspondiente.
4. Serializa el resultado con `joblib.dump`, guardando una tupla con el vectorizador y el clasificador (o vectorizador, SVD y clasificador en el caso del modelo SVD).

Ejecución de la aplicación

Una vez generados los modelos, se lanza la aplicación con:

```
streamlit run app.py
```

Streamlit arranca un servidor local y abre el navegador por defecto en `http://localhost:8501`. La aplicación tiene dos pestañas, accesibles desde la barra lateral:

- **Triaje**: vista del paciente. Permite introducir síntomas por escrito o por voz y elegir el modelo con el que se quiere realizar la clasificación.
- **Administración**: protegida por contraseña. Permite añadir nuevos registros al dataset ampliado, revisar el log de consultas no clasificadas con confianza suficiente, ajustar el umbral de confianza y las rondas de ampliación, y regenerar los modelos.

Configuración del umbral y las rondas

La configuración del comportamiento del triaje se almacena en `config.json`, en la raíz del proyecto. Su contenido por defecto es:

```
{
 "umbral_confianza": 45.0,
 "rondas_extra": 1
}
```

El parámetro `umbral_confianza` indica el porcentaje mínimo de confianza que debe devolver el modelo para considerar válida la predicción. El parámetro `rondas_extra` indica cuántas veces se permite al paciente ampliar sus síntomas antes de pasar al fallback (selección manual o derivación a Medicina General).

Ambos valores se pueden modificar desde la pestaña de administración mediante un formulario, y los cambios se aplican sin necesidad de reiniciar la aplicación.

Flujo de un triaje

Cuando el paciente introduce sus síntomas y pulsa el botón de clasificación, el flujo interno es el siguiente:

1. La aplicación llama a la función del modelo seleccionado (`tf_idf`, `svd`, `svm` o `rna`). Estas funciones cargan el fichero `.joblib` correspondiente con `joblib.load` y devuelven una tupla `(especialidad, confianza)`.

2. Si la confianza supera el umbral configurado, se muestra el resultado directamente.

3. Si la confianza es inferior y todavía quedan rondas de ampliación, la aplicación pide al paciente que añada más síntomas y vuelve a invocar al mismo modelo con la nueva descripción.

4. Si tras agotar las rondas la confianza sigue siendo insuficiente, se entra en la fase de *fallback*: el paciente puede aceptar la sugerencia del modelo, seleccionar manualmente otra especialidad de la lista o quedarse con Medicina General como opción por defecto. En todos los casos, la entrada queda registrada en `logs/triaje_fallback_log.csv`.

Entrada por voz

La aplicación permite dictar los síntomas en lugar de escribirlos. Para ello se utiliza el componente `streamlit-mic-recorder` [2], que captura el audio desde el navegador y devuelve una transcripción de texto en español. Esa transcripción se inserta automáticamente en el campo de síntomas, de manera que el flujo posterior es exactamente el mismo que si el paciente hubiera escrito el texto.

El uso del micrófono requiere que el navegador conceda permiso de acceso. La primera vez que se utiliza, se muestra una ventana de confirmación. En Chrome y en Firefox el permiso queda guardado para futuras sesiones siempre que se acceda desde `localhost`.

D.5. Pruebas del sistema

Dado que el proyecto es un prototipo académico que se ejecuta en local y cuya interfaz cambia con frecuencia durante el desarrollo, se ha optado por una estrategia de pruebas mixta: pruebas manuales sobre la interfaz Streamlit y pruebas funcionales sobre los modelos. No se ha implementado una batería de tests automatizados con `pytest` dentro del alcance del TFG, aunque se considera una mejora futura natural.

Pruebas de los modelos durante el entrenamiento

La evaluación principal de los cuatro modelos se realiza durante el propio proceso de entrenamiento. Los scripts `generarModeloSVM.py` y `generarModeloRNA.py` utilizan GridSearchCV con validación cruzada estratificada de cinco particiones, lo que ofrece una estimación robusta del rendimiento de cada combinación de hiperparámetros sin necesidad de mantener un conjunto de test fijo.

Las métricas calculadas son las habituales en clasificación supervisada multiclase: *accuracy*, *precision*, *recall* y *F1-macro*. La elección de la versión macro frente a la versión micro responde al desequilibrio entre clases del dataset: hay muchos más ejemplos de atención general y medicina interna que de especialidades concretas como dermatología o neurología, y el macro da el mismo peso a todas las clases.

Los modelos TF-IDF base y SVD se utilizan como referencia. Por eso se entrenan con una configuración fija, sin búsqueda exhaustiva, y se evalúan únicamente con la división entrenamiento/test que aplica `train_test_split` con un 20 % de datos reservados.

Pruebas manuales sobre la interfaz

Las funcionalidades de la aplicación se han probado de forma manual a lo largo de todo el desarrollo, siguiendo una serie de escenarios representativos:

- **Triaje con confianza alta**: introducción de descripciones claras (por ejemplo, *"dolor torácico opresivo que irradia al brazo izquierdo"*) para verificar que el sistema devuelve la especialidad esperada por encima del umbral.
- **Triaje con confianza baja**: introducción de descripciones genéricas o ambiguas (*"me encuentro mal"*, *"dolor"*) para comprobar que la

aplicación entra correctamente en la fase de ampliación de síntomas y, después, en la fase de *fallback*.

- **Cambio de modelo**: ejecución de la misma descripción con los cuatro modelos para verificar que cada uno devuelve su propia predicción y porcentaje de confianza.
- **Entrada por voz**: dictado de síntomas en español para comprobar que la transcripción se inserta en el campo de texto y que el flujo posterior funciona igual que con entrada por teclado.
- **Acceso de administrador**: intentos de acceso con credenciales correctas e incorrectas, comprobando que la pestaña permanece bloqueada hasta validar la contraseña.
- **Añadir un nuevo registro**: alta de pares síntoma-especialidad desde la pestaña de administración, verificando que se asigna el siguiente `record_id` disponible y que el registro queda en el CSV ampliado.
- **Revisión del log**: lectura del log de fallback, asignación manual de una especialidad a una entrada y volcado al CSV ampliado. Se ha comprobado que las entradas revisadas se incorporan correctamente al fichero `triajes_baja_confianza_revisados.csv`.
- **Regeneración de modelos**: ejecución de los cuatro botones de regenerado desde la pestaña de administración, verificando que cada script termina correctamente y que la aplicación carga los nuevos `.joblib` sin necesidad de reiniciarse.

Validación cruzada con casos reales

Además de los escenarios anteriores, durante el desarrollo se ha utilizado un pequeño conjunto de descripciones recogidas a partir de testimonios reales (familiares, amigos y consultas habituales) para comprobar el comportamiento del sistema frente a lenguaje coloquial, faltas de ortografía y expresiones poco precisas. Estos casos no forman parte del conjunto de entrenamiento y permiten valorar de forma más realista la capacidad de generalización de los modelos.

Algunas observaciones recogidas durante estas pruebas:

- Las descripciones cortas o muy genéricas tienden a obtener una confianza baja, lo que activa correctamente la ronda de ampliación.

- Los modelos basados en bigramas (SVM y MLP) suelen comportarse algo mejor frente a expresiones clínicas como *"dolor de cabeza intenso"* o *"falta de aire al subir escaleras"*.
- Las negaciones (*"no tengo fiebre"*) no siempre se interpretan bien, ya que la representación de bolsa de palabras no captura ese matiz. Esta limitación es conocida y se asume como parte del alcance del proyecto.

Pruebas de robustez del log y los CSV

También se han probado situaciones límite relacionadas con la gestión de ficheros:

- Ejecución de la aplicación con el log de fallback vacío o inexistente. La aplicación debe crearlo con su cabecera al registrar la primera entrada.
- Síntomas con caracteres no ASCII (tildes, ñ, signos de puntuación). Se ha utilizado codificación Latin-1 al escribir en el dataset y UTF-8 en el log de fallback, comprobando que los caracteres se conservan correctamente en ambos casos.
- Sustitución del fichero `config.json` por un valor mal formado o ausente. La aplicación recurre a los valores por defecto (`umbral_confianza = 60.0`, `rondas_extra = 1`) sin lanzar excepciones.

Posibles mejoras en las pruebas

Como línea de trabajo futura, sería razonable incorporar:

- Una batería de tests automatizados con `pytest` sobre las funciones de preprocesado y carga de modelos, que son fácilmente aislables del resto de la aplicación.
- Un script de evaluación que mantenga un conjunto de test fijo y genere una matriz de confusión por modelo, de manera que la comparación entre versiones del dataset sea reproducible.
- Integración con un servicio externo de análisis estático como SonarCloud para revisar la calidad del código en cada *push* al repositorio.

Estas mejoras no se han incluido en el alcance del TFG por motivos de tiempo, pero el código está estructurado de manera que su incorporación posterior no requiere refactorizaciones importantes.

Apéndice E

Documentación de usuario

E.1. Introducción

Este apéndice está pensado para quien va a utilizar la aplicación, no para quien la programa o la mantiene. Su propósito es doble: explicar qué hace falta tener en el equipo antes de empezar y, una vez instalada, mostrar paso a paso cómo se trabaja con ella.

La aplicación es un prototipo de triaje médico que recibe síntomas escritos o dictados por voz y devuelve la especialidad médica que mejor encaja con la descripción del paciente. Funciona en local, dentro del navegador, y no envía datos a ningún servicio externo. Cuenta con dos vistas diferenciadas: una para el paciente, en la que se introducen los síntomas y se obtiene la especialidad recomendada, y otra para el administrador, en la que se ajusta el comportamiento del sistema, se revisan los casos dudosos y se reentrenan los modelos.

A lo largo del apéndice se describen primero los requisitos que debe cumplir el equipo, después el procedimiento de instalación y, por último, el manual propiamente dicho, organizado por pantallas para que sea fácil seguirlo con la aplicación abierta delante.

E.2. Requisitos de usuarios

La aplicación se ha pensado para ejecutarse en un equipo personal sin necesidad de servidores externos ni conexión permanente a internet, aunque sí hace falta tener instaladas algunas herramientas básicas. Quien la utilice como paciente solo necesita un navegador y, opcionalmente, un micrófono si

quiere dictar los síntomas en lugar de escribirlos. La parte de administración no requiere nada adicional, más allá de conocer la contraseña configurada en el equipo.

Requisitos del sistema operativo

La aplicación se ha probado en Windows 10 y Windows 11, pero también funciona en distribuciones recientes de Linux y en macOS, ya que toda la pila tecnológica (Python, Streamlit, scikit-learn) es multiplataforma. No se requiere ninguna versión concreta del sistema operativo más allá de tener soporte para Python 3.10 o superior.

Requisitos de software

Para poder ejecutar el prototipo en el equipo es necesario disponer del siguiente software previamente instalado:

- **Python 3.10 o superior**. Durante el desarrollo se ha trabajado con la versión 3.11. Se puede descargar desde `https://www.python.org/downloads/`. En Windows conviene marcar la opción *Add Python to PATH* durante la instalación.
- **Pip**, el gestor de paquetes de Python, que viene incluido en la instalación estándar y servirá para instalar las dependencias de la aplicación.
- **Git**, opcional pero recomendable, para descargar el código desde GitHub. Si no se quiere instalar Git, también es posible descargar el proyecto como un archivo ZIP desde la propia interfaz web del repositorio.
- **Un navegador web moderno**: Google Chrome, Mozilla Firefox o Microsoft Edge en versiones recientes. La aplicación se abre automáticamente como una pestaña dentro del navegador.

Requisitos de hardware

El prototipo está diseñado siguiendo el enfoque de IA Frugal, lo que significa que no necesita una máquina potente para funcionar. Los requisitos mínimos recomendados son los siguientes:

- Procesador x86_64 de 64 bits, equivalente a un Intel Core i3 o superior.

- Memoria RAM de 4 GB como mínimo, recomendable 8 GB cuando se vayan a reentrenar los cuatro modelos en una sola sesión.
- Espacio en disco de aproximadamente 500 MB, contando el código fuente, las dependencias instaladas y los ficheros con los modelos entrenados.
- Micrófono integrado o externo, solo si se desea utilizar la entrada por voz. La aplicación funciona igualmente sin él, escribiendo los síntomas directamente.

No es necesaria una tarjeta gráfica dedicada ni soporte para CUDA, ya que los modelos utilizados (TF-IDF, SVD, SVM y MLP) se entrenan y ejecutan sobre CPU sin problemas.

Requisitos de red

La aplicación no necesita conexión a internet para funcionar una vez instalada. La conexión es necesaria únicamente en dos momentos puntuales: la primera descarga del repositorio desde GitHub y la instalación inicial de las dependencias de Python con `pip install`. A partir de ese momento, todo se ejecuta en local y los síntomas del paciente no salen del equipo.

E.3. Instalación

La instalación se compone de cuatro pasos: descargar el código, preparar un entorno virtual de Python, instalar las dependencias y comprobar que los modelos están disponibles. El procedimiento es similar en Windows, Linux y macOS, aunque los comandos varían ligeramente. En este manual se utilizan los comandos de Windows, que son los habituales del entorno de desarrollo.

Descarga del código

El proyecto está alojado en un repositorio público de GitHub. Para descargarlo, basta con clonar el repositorio mediante el comando:

```
git clone https://github.com/aos0020/TFG_AOS0020---TEST.git
```

Alternativamente, si no se tiene Git instalado, se puede descargar el código como un archivo ZIP desde la propia página del repositorio, pulsando

en el botón *Code* y eligiendo la opción *Download ZIP*. Una vez descargado el ZIP, hay que descomprimirlo en una carpeta accesible del equipo.

Para el resto del manual se asume que el código está disponible en una carpeta llamada `TFG_AOS0020--TEST`.

Creación del entorno virtual

Es muy recomendable crear un entorno virtual de Python para no mezclar las dependencias del proyecto con las del resto del sistema. Desde la línea de comandos, situándose dentro de la carpeta del proyecto, se ejecuta:

```
python -m venv .venv
.\.venv\Scripts\activate
```

En Linux o macOS, el segundo comando se sustituye por `source .venv/bin/activate`. Cuando el entorno está activo, el indicador de la consola muestra el prefijo `(.venv)`, lo que confirma que cualquier paquete que se instale a partir de ese momento queda dentro del entorno.

Instalación de dependencias

Con el entorno virtual activo, se instalan todas las librerías necesarias mediante:

```
pip install -r requirements.txt
```

Este paso descarga e instala scikit-learn, pandas, numpy, Streamlit, NLTK, joblib y streamlit-mic-recorder, entre otras. Dependiendo de la conexión y de si las dependencias ya estaban cacheadas, puede llevar entre uno y cinco minutos.

Configuración de la contraseña de administración

La pestaña de administración está protegida por contraseña. Esta contraseña no se distribuye junto con el código, sino que cada usuario debe configurarla en su propio equipo. Para ello, hay que crear el fichero `.streamlit/secrets.toml` dentro de la carpeta del proyecto, con el siguiente contenido:

```
admin_password = "la_contraseña_que_se_quiera"
```

Si este fichero no existe, la aplicación arranca igualmente, pero la pestaña de administración no será accesible. El detalle técnico del mecanismo se explica en el Apéndice D.

Verificación de los modelos entrenados

El repositorio incluye los cuatro modelos ya entrenados como ficheros `.joblib` dentro de las carpetas `TF_IDF/`, `SVD/`, `SVM/` y `RNA/`. Si por algún motivo se quisiera regenerar alguno de ellos antes del primer arranque, basta con ejecutar el script de generación correspondiente, por ejemplo:

```
python TF_IDF\generarModeloTFIDF.py
```

Este paso no es obligatorio en una instalación nueva. Su utilidad principal aparece cuando el administrador, desde la propia aplicación, decide reentrenar los modelos tras haber añadido nuevos casos al conjunto de datos.

Arranque de la aplicación

Una vez instaladas las dependencias y configurada la contraseña, la aplicación se arranca desde la raíz del proyecto con el comando:

```
streamlit run app.py
```

Streamlit abre automáticamente el navegador por defecto en la dirección `http://localhost:8501`. Si no lo hace, basta con copiar esa dirección manualmente en la barra del navegador. Para detener la aplicación, se cierra la pestaña del navegador y se pulsa `Ctrl + C` en la consola donde se está ejecutando Streamlit.

E.4. Manual del usuario

Una vez arrancada la aplicación, la barra lateral izquierda ofrece dos vistas: *Paciente* y *Administrador*. La primera es la que utilizan los usuarios finales para obtener una especialidad a partir de sus síntomas. La segunda es de uso interno y queda reservada al perfil que mantiene el sistema.

Vista del paciente

La pantalla principal de la vista del paciente es la puerta de entrada al triaje. En ella aparece el título *Triaje médico inteligente*, un recordatorio del umbral de confianza y de las rondas de ampliación configuradas, y los controles necesarios para introducir los síntomas y elegir el modelo de clasificación.

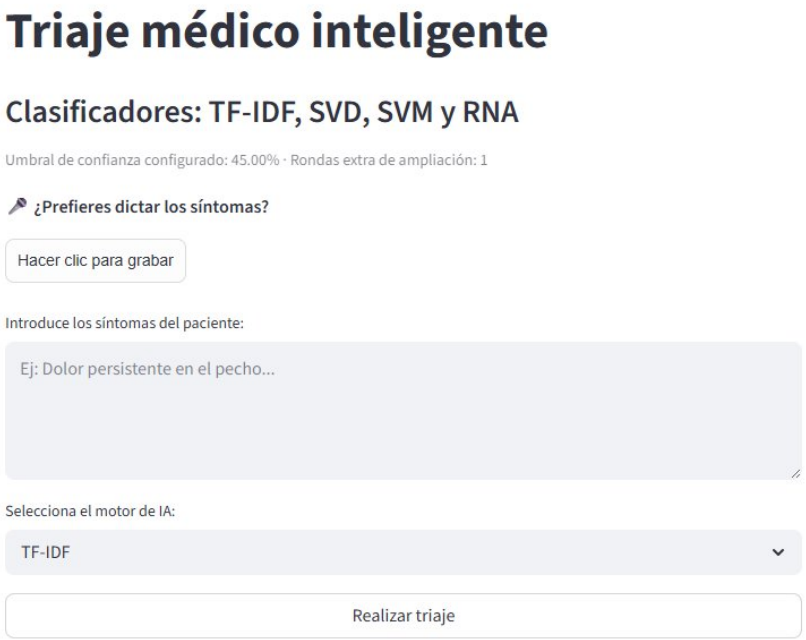

Figura E.1: Pantalla principal del paciente

0.85

Introducción de síntomas

El paciente dispone de dos formas equivalentes de introducir su descripción:

- **Por escrito**, utilizando el cuadro de texto *Introduce los síntomas del paciente*. Es la opción habitual y no requiere ningún hardware adicional. El texto admite frases completas en lenguaje natural, como *me duele mucho la cabeza desde ayer y tengo náuseas*.

- **Por voz**, pulsando el botón *Hacer clic para grabar* situado bajo el icono del micrófono. Al pulsarlo, el navegador solicita permiso para acceder al micrófono. Cuando el usuario habla, la grabación se transcribe automáticamente a texto y se inserta en el mismo cuadro de síntomas, donde puede revisarse antes de continuar.

Las dos vías conducen al mismo punto: el contenido del cuadro de texto es lo que la aplicación utiliza para realizar la clasificación. El dictado por voz es simplemente una alternativa al teclado.

Selección del motor de clasificación

Antes de lanzar el triaje, el paciente puede elegir el modelo de inteligencia artificial que realizará la clasificación. El desplegable *Selecciona el motor de IA* ofrece las cuatro opciones disponibles: TF-IDF, SVD, SVM y RNA. La opción por defecto es TF-IDF, que es la más rápida y se utiliza como línea base.

En un uso normal de la aplicación, esta elección es transparente para el paciente. Sin embargo, durante las pruebas y la evaluación del prototipo resulta muy útil poder cambiar de modelo sobre el mismo texto y comparar las respuestas.

Lanzamiento del triaje

Cuando el texto está introducido y el modelo seleccionado, basta con pulsar el botón *Realizar triaje*. La aplicación procesa internamente la entrada (limpieza, vectorización y clasificación) y muestra el resultado en la misma pantalla. Pueden darse tres situaciones distintas, que se describen a continuación.

Caso 1: confianza suficiente

Cuando la confianza del modelo supera el umbral configurado (por defecto, 45 %), la aplicación muestra directamente la especialidad recomendada junto con el porcentaje de acierto estimado. Este es el caso más habitual y resuelve el triaje en un único paso.

Caso 2: confianza insuficiente y ampliación de síntomas

Si el modelo no alcanza el umbral, la aplicación no descarta el caso. En su lugar, avisa al paciente de que la confianza es baja y le pide que

añada más información sobre lo que está sintiendo. La figura E.2 muestra esta situación con un ejemplo: el modelo TF-IDF sugiere *Medicina Interna* con un 27,75 % de confianza, por debajo del umbral del 45 %, y el sistema solicita una ronda adicional.

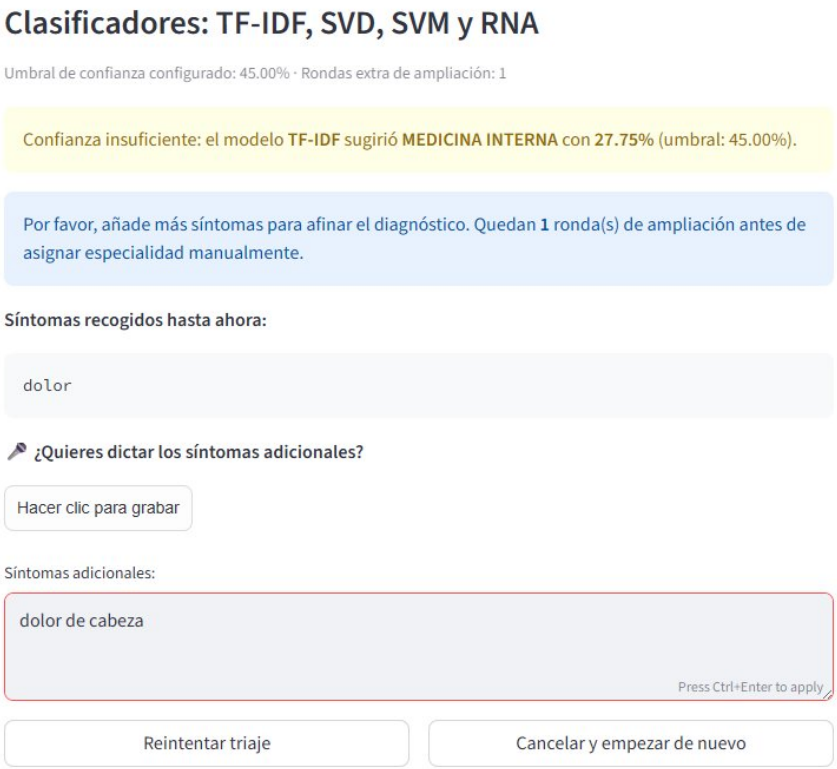

Figura E.2: Ampliación de síntomas tras una predicción de baja confianza

0.85

En esta pantalla, el paciente puede:

- Consultar los *síntomas recogidos hasta ahora*, que se muestran en la parte superior para que pueda ver qué ha entendido el sistema.
- Añadir más síntomas en el cuadro *Síntomas adicionales*, escribiéndolos o dictándolos con el mismo botón de grabación que en la pantalla inicial.
- Pulsar *Reintentar triaje* para volver a lanzar la clasificación con la descripción ampliada.

- Pulsar *Cancelar y empezar de nuevo* si prefiere descartar el caso actual y comenzar desde cero.

El número de rondas de ampliación permitidas se configura desde la vista del administrador. En el caso por defecto se realiza una única ronda.

Caso 3: confianza insuficiente tras las rondas de ampliación

Si después de agotar las rondas de ampliación la confianza sigue por debajo del umbral, la aplicación deja la decisión final en manos del paciente. La figura E.3 muestra este desenlace.

Figura E.3: Selección manual de especialidad tras agotar las rondas

0.85

El paciente recibe un mensaje en el que se indica que el modelo no ha conseguido alcanzar la confianza necesaria, junto con la última especialidad sugerida y su porcentaje. A continuación, dispone de dos opciones equivalentes:

- **Opción 1: aceptar la sugerencia del sistema**, pulsando el botón *Aceptar* seguido del nombre de la especialidad propuesta. Es la opción rápida cuando el paciente cree que la sugerencia es razonable.
- **Opción 2: elegir manualmente otra especialidad** desde el desplegable *Especialidad sugerida por el paciente*. La opción por defecto es *Medicina General*, que actúa como derivación a atención general cuando no hay una orientación clara.

Una vez confirmada la selección con el botón *Confirmar selección*, el caso se registra en un fichero de log para que el administrador pueda revisarlo después. Este registro es la base del mecanismo de aprendizaje continuo del sistema: las consultas que el modelo no ha sabido resolver con seguridad se convierten en nuevos ejemplos de entrenamiento.

Vista del administrador

La pestaña de administración requiere introducir la contraseña configurada en `secrets.toml` durante la instalación. Una vez autenticado, el administrador accede a un conjunto de herramientas pensadas para mantener el sistema y mejorar su comportamiento con el tiempo.

Configuración del triaje

Desde esta sección se ajustan los dos parámetros que controlan el comportamiento del clasificador frente a casos dudosos. La figura E.4 muestra los controles disponibles.

Figura E.4: Configuración del umbral de confianza y de las rondas de ampliación

0.7

Los parámetros que se pueden modificar son los siguientes:

- **Umbral mínimo de confianza (%)**, ajustable mediante un deslizador con valores entre 0 y 100. Es el porcentaje mínimo que debe alcanzar el modelo para que su predicción se acepte directamente, sin pedir más información al paciente. Un valor alto hace al sistema más prudente, ya que pasa por la ampliación más casos; un valor bajo lo hace más resolutivo, pero también más arriesgado.
- **Rondas extra de ampliación de síntomas**, que indica cuántas veces se pedirá al paciente que añada información antes de derivar el caso a selección manual. Por defecto se realiza una única ronda.

Los cambios se aplican pulsando el botón *Guardar configuración* y afectan a todas las consultas posteriores.

Registro de triajes de baja confianza

Esta sección es probablemente la más importante de la pestaña de administración. En ella se muestran los casos en los que el sistema no ha conseguido clasificar con suficiente confianza y que han quedado registrados en el fichero `logs/triaje_fallback_log.csv`. La figura E.5 muestra un ejemplo de la pantalla.

Figura E.5: Revisión y reasignación manual de casos de baja confianza

0.85

Para cada caso registrado, se muestra la fecha y hora, el modelo utilizado y el texto completo de síntomas tal como lo introdujo el paciente. El administrador puede entonces:

- Elegir manualmente la especialidad correcta en el desplegable *Especialidad*.
- Pulsar el botón *Añadir a datos* para volcar ese caso, ya etiquetado, al fichero ampliado `Datos/triajes_baja_confianza_revisados.csv`.

Así, los casos que en su momento fueron problemáticos para el clasificador se convierten en nuevos ejemplos de entrenamiento que servirán para mejorar el sistema cuando se reentrene. Especialidades poco representadas en el conjunto original van ganando ejemplos a medida que se revisan, lo que ayuda a corregir el desequilibrio inicial entre clases.

Alta de nuevos casos

Además del flujo basado en el log, el administrador puede añadir manualmente nuevos pares de síntomas y especialidad. Esta funcionalidad resulta útil cuando se quieren incorporar casos extraídos de fuentes externas, por ejemplo un manual clínico o un protocolo asistencial, sin necesidad de pasar por la vista del paciente.

Los nuevos registros se almacenan en el mismo CSV ampliado que los casos revisados desde el log, manteniendo intacto el dataset original. Esta separación es importante: el fichero base nunca se modifica, lo que permite reproducir cualquier experimento sobre los datos iniciales.

Reentrenamiento de los modelos

Cuando se han añadido suficientes casos nuevos, el administrador puede lanzar el reentrenamiento de los cuatro modelos desde la propia interfaz, sin tocar la línea de comandos. Esta operación lee el CSV original junto con el CSV ampliado, reentrena los cuatro clasificadores (TF-IDF, SVD, SVM y MLP) y sobrescribe los ficheros `.joblib` de cada carpeta de modelo.

Hay que tener en cuenta que el reentrenamiento, especialmente del MLP, puede tardar varios minutos en función del equipo y del tamaño del conjunto ampliado. Durante ese tiempo, la aplicación sigue siendo utilizable, pero los nuevos modelos no estarán activos hasta que el proceso termine y se recargue la página.

Apéndice F

Anexo de sostenibilización curricular

F.1. Introducción

Este anexo incluirá una reflexión personal del alumnado sobre los aspectos de la sostenibilidad que se abordan en el trabajo. Se pueden incluir tantas subsecciones como sean necesarias con la intención de explicar las competencias de sostenibilidad adquiridas durante el alumnado y aplicadas al Trabajo de Fin de Grado.

Más información en el documento de la CRUE https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sosteniblidad_Crue2012.pdf.

Este anexo tendrá una extensión comprendida entre 600 y 800 palabras.

Siglas

MLP *Multi-layer Perceptron*. 1, 6, 7, 11, 14, 31, 43, 46, 55, 59, 68

PLN Procesamiento de Lenguaje Natural. 4, 6, 12, 13

RNA Red Neuronal Artificial. 1, 7, 11, 14, 19, 31, 32, 38, 63

SVD *Singular Value Decomposition*. 1, 5, 6, 11, 12, 13, 14, 19, 31, 32, 33, 36, 38, 59, 63, 68

SVM *Support Vector Machine*. 1, 5, 6, 11, 12, 14, 19, 31, 32, 33, 36, 38, 50, 55, 59, 63, 68

TF-IDF *Term Frequency – Inverse Document Frequency*. 1, 4, 5, 6, 11, 12, 13, 14, 19, 31, 32, 33, 36, 38, 46, 59, 63, 64, 68

Glosario

early stopping Técnica de regularización en el entrenamiento de redes neuronales. Consiste en detener el entrenamiento cuando el error de validación deja de mejorar durante un número determinado de épocas, aunque el error de entrenamiento siga bajando. Evita que el modelo memorice los datos de entrenamiento y pierda capacidad de generalizar. 14

fallback Flujo alternativo que se ejecuta cuando la predicción del modelo no alcanza la confianza mínima exigida. El sistema pide más síntomas durante un número configurable de rondas y, agotadas estas, deja que el paciente acepte la sugerencia del modelo o elija manualmente otra especialidad. 12, 15

GridSearchCV Búsqueda de rejilla con validación cruzada (*Grid Search Cross-Validation*). Procedimiento de exploración estructurada sobre un conjunto discreto de hiperparámetros para encontrar la combinación óptima. 50, 53

standalone Sistema o aplicación que funciona de forma autónoma, sin depender de servicios externos, conexión a internet ni infraestructura adicional. 9, 11, 16, 31, 37

Stemming Proceso lingüístico de reducción de una palabra a su raíz o tallo (*stem*). Técnica de bajo coste computacional comparada con la lematización. 13, 50

TruncatedSVD Variante de la Descomposición en Valores Singulares que opera directamente sobre matrices dispersas sin centrarlas, utilizada

en NLP para reducción de dimensionalidad sobre matrices TF-IDF. 4, 5, 13, 43, 46

Bibliografía

[1] Agencia Tributaria. Retenciones sobre rendimientos del trabajo. tablas de retención IRPF, 2026. [Internet; consultado mayo-2026].

[2] R. Bapt. streamlit-mic-recorder: Streamlit component for audio recording and speech-to-text. https://pypi.org/project/streamlit-mic-recorder/, 2024. [Internet; consultado 13-mayo-2026].

[3] BOE. XVIII Convenio Colectivo Estatal de Empresas de Consultoría, Tecnologías de la Información y Estudios de Mercado y de la Opinión Pública, 2023. [Internet; consultado mayo-2026].

[4] GitHub, Inc. Github: Let's build from here. https://github.com/, 2026. [Internet; consultado 13-mayo-2026].

[5] scikit-learn developers. scikit-learn: Machine learning in Python, 2026. [Internet; consultado mayo-2026].

[6] Seguridad Social. Bases y tipos de cotización 2026, 2026. [Internet; consultado mayo-2026].

[7] Snowflake Inc. Streamlit: A faster way to build and share data apps. https://streamlit.io/, 2026. [Internet; consultado 13-mayo-2026].

[8] Streamlit Inc. Streamlit documentation, 2026. [Internet; consultado mayo-2026].

[9] Zube. Zube — agile project management for GitHub, 2026. [Internet; consultado mayo-2026].